**A PRELIMINARY REPORT ON**

# AI-BASED MEDICAL REPRESENTATIVE VISIT PRIORITIZATION AND ROUTE OPTIMIZATION SYSTEM

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

## BACHELOR OF ENGINEERING (ARTIFICIAL INTELLIGENCE & DATA SCIENCE)

SUBMITTED BY


| | |
|---|---|
| **Bhakti Jayantilal Gundecha** | **Exam No.: 3071** |
| **Parth Lokhande** | **Exam No.: 1020** |
| **Udaykiran Bitla** | **Exam No.: 3056** |

Under the guidance of

**Prof. M. U. Buchade**

## DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

**PUNE VIDYARTHI GRIHA'S COLLEGE OF ENGINEERING, TECHNOLOGY & MANAGEMENT**
**44, VIDYANAGARI, PARVATI, PUNE 411009**


**ACADEMIC YEAR 2026–2027**

**SAVITRIBAI PHULE PUNE UNIVERSITY**

# PVG'S COLLEGE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, PUNE

**Department of Artificial Intelligence & Data Science**

## CERTIFICATE

This is to certify that the preliminary project report entitled **"AI-BASED MEDICAL REPRESENTATIVE VISIT PRIORITIZATION AND ROUTE OPTIMIZATION SYSTEM"**, submitted by **Bhakti Jayantilal Gundecha** (Exam No. 3071), **Parth Lokhande** (Exam No. 1020), and **Udaykiran Bitla** (Exam No. 3056), is a record of the work proposed and documented by them under the guidance of **Prof. M. U. Buchade**. The report is submitted in partial fulfilment of the requirements of Savitribai Phule Pune University for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Data Science.

| | |
|---|---|
| Prof. M. U. Buchade<br>Internal Guide<br>Department of AI & DS | Dr. Mrs. M. P. Atre<br>Head of Department<br>Department of AI & DS |

Dr. S. R. Patil
I/C Principal
PVG's College of Engineering, Technology & Management, Pune

Place: Pune Date: _____

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our internal guide, Prof. M. U. Buchade, for the guidance and feedback provided during the preparation of this preliminary project report. The discussions and review suggestions helped us define the project scope, separate prediction from route optimization, and document the design in a structured manner.

We are thankful to Dr. Mrs. M. P. Atre, Head of the Department of Artificial Intelligence and Data Science, and the faculty members of PVG's College of Engineering, Technology & Management, Punefor their academic support and for providing the opportunity to undertake this project as part of the B.E. programme.

We also acknowledge the authors and maintainers of the research papers, technical documentation, libraries, and optimization tools referenced in this report. Their published work provides the theoretical and practical foundation for machine learning, vehicle routing with time windows, and web-based decision-support systems.

Bhakti Jayantilal Gundecha
Parth Lokhande
Udaykiran Bitla
(B.E. Artificial Intelligence & Data Science)

# ABSTRACT

Medical Representatives (MRs) must visit several doctors within a limited working day. Each planned visit can differ in priority, appointment availability, duration, geographical location, previous visit outcome, and follow-up requirement. Manual planning and spreadsheet-based schedules can therefore lead to avoidable travel, waiting time, missed appointments, and limited visibility for managers. This report presents the design of an AI-based web application that integrates visit-priority prediction with constraint-based route optimization in one decision-support workflow.

The proposed system collects doctor master data, MR profiles, appointment records, historical visits, visit outcomes, and location information. After validation and preprocessing, a supervised machine-learning pipeline is intended to estimate the priority of each planned visit. A Random Forest classifier is proposed as the baseline model, with the final classification or ranking formulation to be confirmed after inspecting the quantity, consistency, and reliability of the available labels. The predicted priority is not used to generate a route by itself. Instead, it becomes one input to a routing engine that considers appointment time windows, service duration, MR working hours, start and end locations, and travel-time estimates. Google OR-Tools is proposed for solving a Vehicle Routing Problem with Time Windows (VRPTW). The result is a daily visit sequence with estimated timing and travel information.

The application is planned with React.js for the user interface, FastAPI and Python for backend services, scikit-learn for machine learning, Google OR-Tools for routing, and MySQL for persistence. Role-based interfaces are designed for MRs and managers. The report documents the problem definition, related work, software requirements, architecture, data model, UML diagrams, project plan, risks, security considerations, and evaluation methodology. Since implementation and experiments are still in progress, numerical claims of model accuracy or route improvement are not made. Evaluation will compare the proposed approach against a manual or simple baseline using prediction metrics and route-quality measures.



**Keywords:** Medical Representative, Visit Prioritization, Machine Learning, Random Forest, Route Optimization, VRPTW, Google OR-Tools, Decision Support System.

# CONTENTS

**ACKNOWLEDGEMENT** ii

**ABSTRACT** iii

**LIST OF ABBREVIATIONS** xiv

**NOMENCLATURE** xv

**1 INTRODUCTION** 1

1.1 Project Idea . . . . . 1

1.2 Motivation of the Project . . . . . 1

1.3 Background and Context . . . . . 1

1.4 Problem Idea and Existing Practice . . . . . 2

1.5 Motivation for Integrating Prediction and Optimization . . . . . 2

1.6 Organization of the Report . . . . . 3

**2 LITERATURE SURVEY** 4

2.1 Introduction . . . . . 4

2.2 Review of Related Work . . . . . 4

2.2.1 Classical Vehicle Routing . . . . . 4

2.2.2 Vehicle Routing with Time Windows . . . . . 5

2.2.3 Dynamic and Time-Dependent Routing . . . . . 5

2.2.4 Physician Detailing and Sales Representative Planning . . . . . 6

2.2.5 Machine Learning and Random Forests . . . . . 6

2.2.6 Evaluation Methods . . . . . 7

2.3 Summary of Surveyed Literature . . . . . 7

2.4 Research Gap . . . . . 10

**3 PROBLEM DEFINITION AND SCOPE** **11**

3.1 Problem Statement . . . . . 11

3.2 Goals and Objectives . . . . . 11

3.3 Statement of Scope . . . . . 12

3.3.1 Included in the Prototype . . . . . 12

3.3.2 Outside the Initial Scope . . . . . 12

3.3.3 Inputs and Outputs . . . . . 13

3.4 Software Context . . . . . 13

3.5 Major Constraints . . . . . 13

3.6 Methodologies of Problem Solving and Efficiency Issues . . . . . 14

3.7 Multi-Core, Embedded and Distributed Computing Context . . . . . 14

3.8 Expected Outcomes . . . . . 15

3.9 Applications . . . . . 15

3.10 Hardware Resources Required . . . . . 15

3.11 Software Resources Required . . . . . 15

**4 PROJECT PLAN** **17**

4.1 Project Estimates . . . . . 17
4.1.1 Cost Estimate . . . . . 17
4.1.2 Time Estimate . . . . . 18
4.2 Project Resources . . . . . 18
4.2.1 Team Organization . . . . . 19
4.2.2 Management Reporting and Communication . . . . . 19
4.3 Risk Management with Respect to NP-Hard Analysis . . . . . 19
4.3.1 Risk Identification and Analysis . . . . . 19
4.3.2 Risk Mitigation, Monitoring and Management . . . . . 21
4.4 Project Schedule . . . . . 22
4.5 Task Network . . . . . 23

**5 SOFTWARE REQUIREMENTS SPECIFICATION 24**

5.1 Introduction . . . . . 24
5.2 Project Scope . . . . . 24
5.3 User Classes and Characteristics . . . . . 24
5.3.1 Medical Representative . . . . . 24
5.3.2 Manager . . . . . 25
5.3.3 Administrator or Authorized Maintainer . . . . . 25
5.4 Assumptions and Dependencies . . . . . 25
5.5 Functional Requirements . . . . . 25
5.6 External Interface Requirements . . . . . 27
5.6.1 User Interfaces . . . . . 27

5.6.2 Software Interfaces . . . . . 28
5.6.3 Hardware Interfaces . . . . . 28
5.6.4 Communication Interfaces . . . . . 28
5.7 Non-Functional Requirements . . . . . 28
5.7.1 Performance Requirements . . . . . 28
5.7.2 Safety and Reliability Requirements . . . . . 29
5.7.3 Security Requirements . . . . . 29
5.7.4 Software Quality Attributes . . . . . 29
5.8 System Requirements . . . . . 29
5.8.1 Database Requirements . . . . . 29
5.8.2 Software Requirements . . . . . 30
5.8.3 Hardware Requirements . . . . . 30
5.9 SDLC Model to Be Applied . . . . . 30
5.10 System Implementation Plan . . . . . 30

**6 DETAILED DESIGN 32**

6.1 Introduction . . . . . 32
6.2 Architectural Design . . . . . 32
6.2.1 Presentation Layer . . . . . 32
6.2.2 Application and API Layer . . . . . 32
6.2.3 Machine-Learning Layer . . . . . 34
6.2.4 Optimization Layer . . . . . 34
6.2.5 Persistence Layer . . . . . 34

6.3 System Workflow and Data Flow . . . . . 34
6.4 UML Diagrams . . . . . 35
6.4.1 Use Case Diagram . . . . . 35
6.4.2 Activity Diagram: MR User Behaviour . . . . . 35
6.4.3 Sequence Diagram: Route Generation . . . . . 36
6.4.4 Class Diagram . . . . . 37
6.4.5 Entity–Relationship Diagram . . . . . 37
6.4.6 State Diagram: Route Lifecycle . . . . . 39
6.4.7 Component Diagram . . . . . 39
6.4.8 Deployment Diagram . . . . . 39
6.4.9 Data Design . . . . . 40
6.4.10 Internal, Global, and Temporary Data Structures . . . . . 41
6.4.11 Database Description and Integrity Rules . . . . . 41
6.5 Machine-Learning Priority Model Design . . . . . 42
6.5.1 Feature Set . . . . . 42
6.5.2 Preprocessing and Training . . . . . 42
6.5.3 Random Forest Formulation . . . . . 43
6.5.4 Prediction Output and Fallback . . . . . 43
6.6 Route Optimization Design . . . . . 43
6.6.1 VRPTW Definition . . . . . 43
6.6.2 Illustrative Objective . . . . . 44
6.6.3 Illustrative Appointment Inputs . . . . . 44

6.6.4 Travel-Time Matrix . . . . . 45

6.6.5 Infeasibility Handling . . . . . 45

6.7 Module Design . . . . . 45

6.8 Illustrative Dashboard Behaviour . . . . . 46

6.9 API Design Overview . . . . . 47

**7 OTHER SPECIFICATIONS 48**

7.1 Advantages . . . . . 48

7.2 Limitations . . . . . 48

7.3 Security and Privacy . . . . . 49

7.4 Testing Strategy . . . . . 49

7.5 Evaluation Plan . . . . . 50

7.5.1 ML Evaluation . . . . . 50

7.5.2 Route Evaluation . . . . . 51

7.6 Applications . . . . . 51

7.7 Future Scope . . . . . 51

**8 SUMMARY AND CONCLUSION 53**

8.1 Summary . . . . . 53

8.2 Conclusion . . . . . 53

8.3 Next Steps . . . . . 54

**REFERENCES 55**

**A PROJECT PLANNER** **58**

**B REVIEWER COMMENTS AND ACTION LOG** **59**

**C PLAGIARISM REPORT** **60**

# LIST OF FIGURES

1.1 Problem context for daily MR visit planning. . . . . 2

2.1 Research contribution: integration of priority prediction and feasible routing. 10

4.1 Indicative dependency-aware project timeline (relative weeks, not calendar dates). . . . . 23

4.2 Project task dependency network. . . . . 23

5.1 Iterative development model proposed for the prototype. . . . . 30

6.1 Overall system architecture. . . . . 33

6.2 End-to-end MR data flow. . . . . 35

6.3 Use case UML diagram for the proposed application. . . . . 35

6.4 Activity UML diagram showing MR behaviour and exception paths. . . . 36

6.5 Sequence UML diagram for optimized route generation. . . . . 37

6.6 Conceptual UML class diagram. . . . . 38

6.7 Conceptual entity–relationship diagram. . . . . 38

6.8 State UML diagram for the route lifecycle. . . . . 39

6.9 Component diagram of the proposed software modules. . . . . 39

6.10 Deployment diagram for the academic prototype. . . . . 40

6.11 Proposed priority-prediction workflow. . . . . 42

# LIST OF TABLES

1.1 Existing approach and identified limitations. . . . . 3

2.1 Summary of relevant literature and its relationship to the project. . . . . 7

3.1 Suggested development hardware requirements. . . . . 16

3.2 Proposed software resources. . . . . 16

4.1 Preliminary cost-estimation basis. . . . . 17

4.2 Proposed development estimate. . . . . 18

4.3 Project team and initial coordination plan. . . . . 19

4.4 Project risk analysis. . . . . 20

4.5 Risk mitigation plan. . . . . 21

5.1 Functional requirements and acceptance checks. . . . . 26

6.1 Proposed core data entities. . . . . 40

6.2 Illustrative inputs from the design presentation; not experimental results. . 45

6.3 Functional module breakdown. . . . . 45

6.4 Indicative REST API endpoints. . . . . 47

7.1 Illustrative test plan. . . . . 50

7.2 Baseline and proposed-system comparison framework. . . . . 52

A.1 Project planner checklist. . . . . 58

B.1 Reviewer comment and action log. . . . . 59

# LIST OF ABBREVIATIONS

| | |
|---|---|
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **CRUD** | Create, Read, Update, Delete |
| **DBMS** | Database Management System |
| **ER** | Entity–Relationship |
| **F1** | Harmonic mean of precision and recall |
| **HTTP** | Hypertext Transfer Protocol |
| **ML** | Machine Learning |
| **MR** | Medical Representative |
| **ORM** | Object-Relational Mapping |
| **REST** | Representational State Transfer |
| **ROC-AUC** | Receiver Operating Characteristic Area Under the Curve |
| **SRS** | Software Requirements Specification |
| **UI** | User Interface |
| **UML** | Unified Modeling Language |
| **VRP** | Vehicle Routing Problem |
| **VRPTW** | Vehicle Routing Problem with Time Windows |

# NOMENCLATURE

| | |
|---|---|
| $D$ | Set of doctors or visit locations planned for a day. |
| $V$ | Set of routing nodes, including start/end locations. |
| $A$ | Set of feasible directed arcs between routing nodes. |
| $t_{ij}$ | Estimated travel time from node $i$ to node $j$. |
| $d_{ij}$ | Estimated distance from node $i$ to node $j$. |
| $s_i$ | Service duration for the visit at node $i$. |
| $[a_i, b_i]$ | Permitted appointment time window for visit $i$. |
| $H$ | MR working-hour horizon for the planned day. |
| $x_{ij}$ | Binary decision variable equal to 1 if the route travels directly from $i$ to $j$. |
| $p_i$ | Predicted priority score or class for visit $i$. |
| $u_i$ | Penalty associated with leaving important visit $i$ unserved. |
| $w_t, w_w, w_u$ | Weights for travel time, waiting time, and unserved-visit penalties. |

# CHAPTER 1

# INTRODUCTION

## 1.1 PROJECT IDEA

The project proposes a web-based decision-support system that estimates the importance of planned doctor visits from historical and operational records, then generates a feasible daily visit route subject to appointment and working-hour constraints. It combines predictive modelling with operations research rather than treating doctor selection, scheduling, and navigation as unrelated activities.

## 1.2 MOTIVATION OF THE PROJECT

Medical Representatives perform field work that involves visiting doctors, communicating approved product information, recording visit outcomes, and arranging follow-ups. Their time is limited, and doctors are not available at identical hours or locations. A plan based only on manual judgement may ignore differences in follow-up urgency or generate unnecessary movement between geographically distant locations. A single system that shows why a visit is important and when it can feasibly be performed can support more consistent planning and more transparent monitoring. The intended application is a planning aid: the MR and manager remain responsible for reviewing the schedule and handling business exceptions.

## 1.3 BACKGROUND AND CONTEXT

The project joins two decision problems. Visit prioritization estimates the relative importance of a planned interaction. Route optimization determines a sequence and schedule that respects temporal and operational constraints. These problems are related but are not interchangeable. A high-priority doctor cannot automatically be visited first if

the doctor is unavailable until later, and a short route is not necessarily a good route if it systematically omits important follow-ups.

The proposed workflow uses operational data as input, generates a priority output from a machine-learning model, and supplies that output to a constrained optimizer. The web application exposes the schedule and supporting information to an MR and provides aggregated activity information to a manager. This design is consistent with the separation of travel-cost modeling and time-window constraints demonstrated in established VRPTW literature and the OR-Tools routing examples [3, 20].

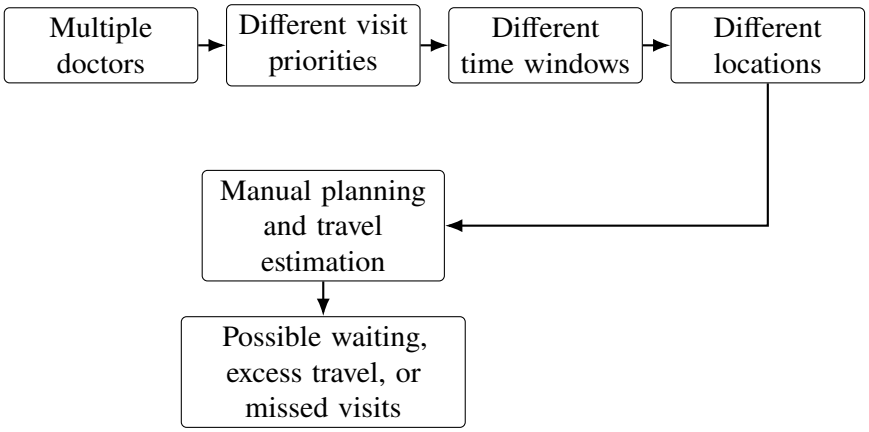



**Figure 1.1:** Problem context for daily MR visit planning.

## 1.4 PROBLEM IDEA AND EXISTING PRACTICE

A typical manual process begins with a list of doctors assigned to a territory. The MR or manager checks appointments, chooses visits, estimates travel, and prepares a daily schedule. Maps may help navigate between two points but do not, by themselves, resolve visit selection, visit duration, appointment windows, and the complete daily route as one optimization problem. Separate appointment records and static schedules also make it difficult to re-plan when an appointment changes.

## 1.5 MOTIVATION FOR INTEGRATING PREDICTION AND OPTIMIZATION

The project is not intended to replace field judgement with a fully autonomous decision. Instead, it provides structured information to help a user decide which visits are important and how they can be scheduled. Machine learning can estimate priority when reliable

| Existing practice | Limitation addressed by the project |
|---|---|
| Manual prioritization | Can vary by individual judgement and may not consistently use historical follow-up evidence. |
| Spreadsheet planning | Becomes difficult to maintain as doctor records, appointment changes, and territories grow. |
| Map navigation | Optimizes navigation between points, not the full appointment-constrained schedule. |
| Separate appointment records | Does not necessarily coordinate priority, service duration, and every stop in the route. |
| Static daily plan | Requires manual changes when appointments or visit outcomes change. |
| Limited analytics | Makes it harder for managers to compare planned, completed, pending, and high-priority visits. |

**Table 1.1:** Existing approach and identified limitations.

historical labels exist. A routing optimizer can then use the output alongside explicit constraints. The separation also makes the system easier to audit: a priority prediction can be inspected independently from a route decision.

## 1.6 ORGANIZATION OF THE REPORT

Chapter 1 introduces the problem and motivation. Chapter 2 reviews related work. Chapter 3 defines the problem, scope, outcomes, and constraints. Chapter 4 presents the project plan and risk management. Chapter 5 specifies functional and non-functional requirements. Chapter 6 documents system, data, and UML design. Chapter 7 summarizes advantages, limitations, and applications. Chapter 8 concludes the report and identifies the next stage of implementation. References and annexures follow the main chapters.

# CHAPTER 2

# LITERATURE SURVEY

## 2.1 INTRODUCTION

Research relevant to this project falls into several overlapping areas: vehicle routing, routing with appointment windows, sales-force and physician-detailing planning, dynamic routing, supervised classification, and software-based decision support. Classical routing work explains why large route spaces need systematic algorithms, while later VRPTW research adds explicit visit windows and improved heuristics. Predictive modelling supplies an estimate of visit importance, but it does not solve the route sequencing problem by itself.

## 2.2 REVIEW OF RELATED WORK

### 2.2.1 Classical Vehicle Routing

Dantzig and Ramser formalized an early truck-dispatching problem and demonstrated the importance of computational methods for routing multiple service points from a depot [1]. Clarke and Wright proposed a savings-based method that constructs economical routes by combining deliveries that would otherwise be served separately [2]. These studies established the principle that route planning can be framed as a combinatorial optimization problem rather than handled only by visual inspection.

Laporte reviewed exact and approximate algorithms for the vehicle routing problem and organized the main families of solution methods [6]. Toth and Vigo later consolidated models, algorithms, and variants in a comprehensive treatment of vehicle routing [5]. This project inherits the routing formulation but adds the particular requirements of a field representative whose visits have time windows, heterogeneous importance, and recorded follow-up outcomes.

### 2.2.2 Vehicle Routing with Time Windows

Solomon studied route construction and scheduling when customer visits must be made within specified time windows. The paper introduced benchmark instances and compared algorithms under different window characteristics [3]. Solomon and Desrosiers surveyed routing and scheduling problems constrained by time windows, emphasizing that temporal constraints affect both feasibility and algorithm design [4].

Bräysy and Gendreau reviewed construction and local-search algorithms for the VRPTW and discussed how heuristic methods should be compared [7]. Their follow-up survey covers metaheuristic methods for the same class of problem [8]. Pisinger and Ropke introduced a general heuristic framework capable of addressing several vehicle-routing variants [9]. These works motivate a baseline-versus-proposed evaluation rather than assuming that a solver improves every route without empirical comparison.

The Google OR-Tools documentation provides a practical formulation of the VRPTW using a time matrix, time-window ranges, vehicle starts and ends, and a routing search strategy [20]. The project uses this as implementation guidance. OR-Tools requires travel costs and time data; it does not create geographically accurate travel times automatically. The source of the distance-time matrix must therefore be declared and evaluated separately.

### 2.2.3 Dynamic and Time-Dependent Routing

Pillac and colleagues reviewed dynamic vehicle-routing problems, distinguishing information available before planning from information arriving during execution [10]. The key implication for an MR workflow is that appointment changes and field events may invalidate a route, so a later iteration should support controlled route regeneration. Bektas and Laporte's work on the pollution-routing problem illustrates how routing objectives can extend beyond distance to operational consequences such as fuel use and emissions [11]. For this academic prototype, the primary focus remains travel time, waiting, visit coverage, and feasibility; environmental measures can be added where reliable data is available.

### 2.2.4 Physician Detailing and Sales Representative Planning

Yi, Anandalingam, and Sorrell developed an expert-system approach to physician-detailing planning that combines knowledge about physician response with neural-network modelling and nonlinear programming [12]. Yi and colleagues later examined a knowledge-based method for planning the sequence of product detailing, with a hybrid of neural networks and nonlinear optimization [13]. These works are particularly relevant because they treat physician interactions as resource-allocation decisions and demonstrate the value of integrating domain information with optimization.

The Review III presentation also identifies related studies by Zhang and colleagues on pharmaceutical sales-representative routing, Rhee and Yi on physician-interest prediction, Bouatouche and Belkadi on sales-representative planning, and Shala and colleagues on field-service planning. The supplied slide deck does not include full paper titles, venues, years, or DOIs for these entries. They are therefore treated here as areas identified during the project review, not as fully verified bibliographic records. Their references should be completed by the project team before final submission.

### 2.2.5 Machine Learning and Random Forests

Breiman introduced Random Forests as an ensemble of tree predictors trained with randomized samples and feature selection [14]. The method is a reasonable baseline for tabular operational data because it can model non-linear relationships and interactions without requiring all features to be transformed into a linear scale. It can also estimate class probabilities, which can be useful if the optimizer needs a graded priority score rather than only a text label.

Pedregosa and colleagues described scikit-learn as a consistent Python machine-learning toolkit with estimators and evaluation tools for common learning tasks [15]. The project proposes a preprocessing pipeline, train/test separation, and a Random Forest baseline. The choice must be revisited if the training data lacks dependable priority labels, is too small, or contains highly imbalanced classes. Under those conditions, a transparent rule-based ranking may be a more defensible initial approach than training a model whose

output cannot be validated.

### 2.2.6 Evaluation Methods

Fawcett explained Receiver Operating Characteristic analysis and its use in evaluating binary classifiers across decision thresholds [16]. Davis and Goadrich discussed the relationship between precision-recall and ROC curves [17]; Saito and Rehmsmeier argued that precision-recall plots are particularly informative in imbalanced classification settings [18]. Accordingly, this report proposes accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC where appropriate. Metrics will be reported by class, and performance on high-priority visits will be inspected specifically instead of relying on accuracy alone.

## 2.3 SUMMARY OF SURVEYED LITERATURE

**Table 2.1:** Summary of relevant literature and its relationship to the project.

| Study | Main focus | Method / Contribution | Relevance or limitation |
|---|---|---|---|
| Dantzig and Ramser [1] | Truck dispatching | Early computational routing formulation | General routing foundation; not MR-specific. |
| Clarke and Wright [2] | Multi-stop routing | Savings heuristic | Useful as a simple baseline; does not predict visit priority. |
| Solomon [3] | VRPTW | Test instances and heuristics | Supports time-window modelling and comparative testing. |

| Study | Main focus | Method / Contribution | Relevance or limitation |
|---|---|---|---|
| Solomon and Desrosiers [4] | Time-window routing | Survey of routing/scheduling problems | Explains how temporal feasibility changes the problem. |
| Laporte [6] | VRP algorithms | Review of exact and approximate methods | Establishes algorithm families and computational trade-offs. |
| Bräysy and Gendreau [7] | VRPTW heuristics | Construction and local search survey | Highlights baseline selection and algorithm comparison. |
| Bräysy and Gendreau [8] | VRPTW metaheuristics | Review of metaheuristic methods | Relevant to difficult constrained instances. |
| Pisinger and Ropke [9] | General VRP variants | Unified heuristic framework | Supports flexible routing formulations. |
| Pillac et al. [10] | Dynamic VRP | Survey of dynamic problems | Motivates replanning after changes. |
| Bektas and Laporte [11] | Pollution routing | Routing with environmental cost | Suggests future objective extensions. |
| Yi et al. [12] | Physician detailing | Neural networks with nonlinear optimization | Domain-specific evidence for integrated planning. |

| Study | Main focus | Method / Contribution | Relevance or limitation |
|---|---|---|---|
| Yi et al. [13] | Detailing-plan quality | Knowledge-based weights and hybrid modelling | Supports combining domain knowledge with prediction. |
| Breiman [14] | Classification/ regression | Random Forest ensemble | Proposed baseline for visit-priority prediction. |
| Pedregosa et al. [15] | ML tooling | scikit-learn library | Practical pipeline and model evaluation support. |
| Fawcett [16] | Classifier evaluation | ROC analysis | Helps assess discrimination across thresholds. |
| Davis and Goadrich [17] | Classifier evaluation | Precision-recall and ROC relationship | Supports suitable metric selection. |
| Saito and Rehmsmeier [18] | Imbalanced learning | Precision-recall evaluation analysis | Relevant if priority classes are imbalanced. |
| Liu et al. [19] | VRPTW literature | Systematic review of recent methods | Shows modern VRPTW work uses complex methods and constraints. |
| Google OR-Tools [20] | Practical routing solver | Time dimension and time-window constraints | Direct reference for the proposed optimization module. |
| React documentation [21] | Frontend UI | Component-based UI development | Supports dashboards and user interaction. |

| Study | Main focus | Method / Contribution | Relevance or limitation |
|---|---|---|---|
| FastAPI documentation [22] | Backend APIs | Python API framework and validation | Supports service separation and request validation. |
| MySQL documentation [23] | Relational persistence | Database engine and SQL features | Supports structured operational data. |

## 2.4 RESEARCH GAP

The reviewed material covers physician targeting, sales-force planning, appointment scheduling, vehicle routing, and field-service optimization, but these themes are commonly formulated with different assumptions and objectives. The project-specific gap is the integration of a transparent visit-priority output with a route optimizer that explicitly checks appointment windows, service duration, MR working hours, and travel time in one MR-facing workflow.

The intended contribution is not a claim of a new routing algorithm. It is an applied system design that links a data-driven priority estimate to a constraint-based route and presents both to operational users. Its value must be demonstrated through evaluation on approved real or synthetic data.

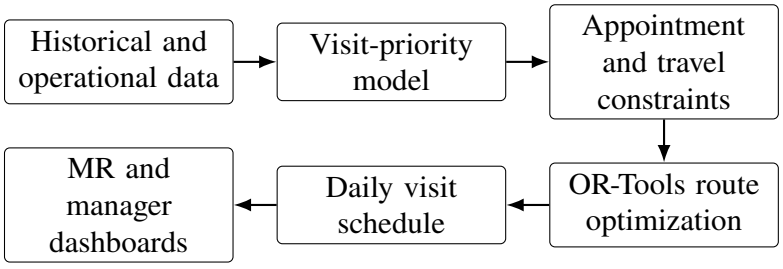



**Figure 2.1:** Research contribution: integration of priority prediction and feasible routing.

# CHAPTER 3

# PROBLEM DEFINITION AND SCOPE

## 3.1 PROBLEM STATEMENT

Medical Representatives must visit multiple doctors during a workday, but each doctor may have a different importance level, availability window, location, service duration, and follow-up requirement. The representative must complete as many valuable visits as possible without violating appointment and working-hour constraints. Manual planning does not consistently integrate all of these factors, while generic map navigation only supports part of the task.

The problem is to design and implement a web-based decision-support system that (i) estimates the priority of planned visits from historical and operational data, (ii) finds a feasible and efficient visit sequence under temporal and travel constraints, and (iii) displays daily planning and activity information to MRs and managers. The quality of the final schedule depends on input quality, travel-time data, the available appointment windows, and the selected optimization objective.

## 3.2 GOALS AND OBJECTIVES

The overall goal is to support data-informed field planning while preserving the ability for users to inspect and override operational decisions. Specific objectives are:

1. Define and validate data entities for doctors, MRs, visits, appointments, predicted priorities, routes, and route stops.
2. Prepare a repeatable data-cleaning and feature-engineering pipeline for historical and operational records.
3. Establish a baseline priority-prediction model, initially a Random Forest classifier

if adequate labels are available.

4. Model working hours, doctor availability, visit duration, appointment windows, and travel times as routing constraints.

5. Use Google OR-Tools to produce a feasible route that balances travel effort and visit value.

6. Provide separate role-based views for MR day planning and manager-level monitoring.

7. Evaluate model quality and route quality against transparent baselines without presuming improvements before experiments.

## 3.3 STATEMENT OF SCOPE

### 3.3.1 Included in the Prototype

The planned prototype includes user authentication and role-based access; doctor and MR master data; appointment and visit management; priority inference; route generation; daily schedule display; visit status updates; and manager dashboard summaries. It will support a limited experimental territory and a bounded daily set of visit candidates appropriate for testing. The optimizer will accept a travel-time matrix produced by a permitted map service, agency-approved data, precomputed routes, or a controlled simulation.

### 3.3.2 Outside the Initial Scope

The initial academic prototype does not require patient medical records, does not guarantee live traffic data, and does not claim integration with a commercial pharmaceutical CRM. Full GPS tracking, mobile application deployment, automated rescheduling with external calendars, and production-grade multi-city scaling are future extensions. Real-time performance and business benefits will not be asserted without experiments.

### 3.3.3 Inputs and Outputs

Major inputs include doctor ID, territory, location coordinates, availability window, typical visit duration, MR working start/end times, start/end locations, planned appointment date and time, previous visit history, visit outcome, follow-up flag, and priority labels where available. Inputs must be validated for missing IDs, invalid coordinates, impossible time windows, invalid durations, and inconsistent relationships.

The system is expected to output the predicted priority category or score, a proposed visit order, estimated arrival/start times, travel time and distance where available, unserved visits with reasons where detectable, and dashboard summaries of planned/completed/pending activity. The schedule is a recommendation subject to review by authorized users.

## 3.4 SOFTWARE CONTEXT

The project is a web-based operational decision-support application. The React frontend communicates with a FastAPI backend through REST endpoints. The backend manages authentication, validation, domain logic, the priority-prediction pipeline, and route optimization. MySQL stores structured entities. scikit-learn provides the model implementation, and Google OR-Tools provides routing search and time-window constraints. Map or routing data is provided by an external service or controlled input dataset, depending on the experiment setup.

## 3.5 MAJOR CONSTRAINTS

1. **Data quality:** Missing historical visits, incomplete labels, invalid coordinates, or inconsistent appointment records can reduce prediction and routing quality.
2. **Label availability:** Supervised learning requires priority labels with a consistent meaning. If the agency cannot supply them, the project must use an explicitly documented rule-based baseline or ranking formulation.
3. **Appointment feasibility:** High priority cannot override a hard appointment window

unless a user explicitly changes the constraint.

4. **Routing complexity:** VRPTW is combinatorial; exact optimality cannot be guaranteed for every practical instance within a useful response time. The system should favour a feasible solution and transparently report infeasibility.

5. **Travel-time uncertainty:** A precomputed matrix may not reflect live traffic or temporary road conditions.

6. **Privacy:** Only data needed for scheduling should be stored; synthetic identifiers should be preferred for academic experiments.

7. **Prototype boundary:** The technology stack and available development time limit initial scale and integration scope.

## 3.6 METHODOLOGIES OF PROBLEM SOLVING AND EFFICIENCY ISSUES

Three options are considered. Manual or spreadsheet planning has a low setup cost but depends heavily on user effort. A simple nearest-neighbour or fixed-priority ordering algorithm is fast and transparent but may create poor schedules under time windows. The proposed pipeline first predicts visit priority and then applies a constrained optimizer. The third option is more involved but allows objective comparison between data-informed priority and a feasible sequence.

Efficiency will be assessed using total travel time, total distance, waiting time, number of completed visits, unserved visits, time-window violations, and computation time. Prediction will be assessed independently using class-sensitive metrics. This separation helps determine whether a weak outcome originates in the ML layer, the travel-time input, or the optimization layer.

## 3.7 MULTI-CORE, EMBEDDED AND DISTRIBUTED COMPUTING CONTEXT

The prototype is a conventional client-server web application and does not require embedded computing. Distributed computing is present in the broad sense that browser-based clients communicate with a backend service and database over a network. The ML and optimization

tasks may run on the backend. Multi-core execution can be used by underlying numerical libraries or server processes, but parallel computation is not a design requirement for the first version. If the system later serves multiple territories or MRs concurrently, task queues and isolated optimization jobs may help manage load.

## 3.8 EXPECTED OUTCOMES

The expected outcome is a documented and testable prototype design with a validated data model, REST interface, priority-prediction pipeline, route-optimization module, MR dashboard, manager dashboard, and evaluation harness. A successful prototype should produce feasible routes for the supported test cases, display any unscheduled visits clearly, protect role-specific records, and provide reproducible baseline comparisons. Quantitative improvements will be reported only after running experiments.

## 3.9 APPLICATIONS

Potential use cases include daily doctor-visit planning for pharmaceutical representatives, territory-based field sales scheduling, prioritization of follow-up visits, manager monitoring of daily field activities, and research on integrating prediction with route optimization. Similar techniques may also apply to non-medical field-service workflows, provided their business rules and privacy requirements are separately modelled.

## 3.10 HARDWARE RESOURCES REQUIRED

## 3.11 SOFTWARE RESOURCES REQUIRED

| No. | Parameter | Suggested minimum | Justification |
|---|---|---|---|
| 1 | CPU | Dual-core 2 GHz or better | Suitable for application development and small test datasets. |
| 2 | RAM | 4 GB minimum; 8 GB recommended | Allows browser, editor, database, and backend services to run together. |
| 3 | Storage | 5 GB free workspace | Source, sample datasets, environment files, and logs. |
| 4 | Display | $1366 \times 768$ or better | Supports development and dashboard testing. |
| 5 | Network | Stable internet access when external APIs are used | Needed for package installation and any external routing service. |

**Table 3.1:** Suggested development hardware requirements.

| Category | Proposed software or tool |
|---|---|
| Operating system | Windows, Linux, or macOS development environment. |
| Frontend | React.js and a modern browser. |
| Backend | Python with FastAPI. |
| Machine learning | pandas/NumPy for data handling and scikit-learn for preprocessing and Random Forest. |
| Optimization | Google OR-Tools routing library. |
| Database | MySQL relational database. |
| API testing | OpenAPI interface exposed by FastAPI and an HTTP client. |
| Version control | Git with a remote repository as available. |
| Diagramming/ documentation | LaTeX/TikZ or an equivalent diagram tool. |

**Table 3.2:** Proposed software resources.

# CHAPTER 4

# PROJECT PLAN

## 4.1 PROJECT ESTIMATES

Project estimates are planning estimates for the academic prototype, not vendor quotations or approved expenditure. The team should update them after the chosen dataset, hosting approach, and any external API requirements are confirmed.

### 4.1.1 Cost Estimate

The design can be developed with open-source languages and libraries. The following plan assumes local development, a local or student-accessible MySQL instance, and public documentation. If a commercial map API, cloud hosting, SMS, or production authentication service is adopted, those costs must be recorded separately.

| Resource | Cost approach | Note |
|---|---|---|
| Development tools | Open-source / existing access | No licence cost assumed in this estimate. |
| Database and backend | Local development | Hosting is a separate future decision. |
| Routing data | Approved/free data or controlled matrix for prototype | API terms, quotas, and billing must be checked before use. |
| Test dataset | Synthetic or approved anonymized data | No patient medical records are required by the core workflow. |
| Deployment | To be determined | No production hosting amount is asserted. |

**Table 4.1:** Preliminary cost-estimation basis.

### 4.1.2 Time Estimate

The project can be managed in phases. The schedule below is a proposed sequence rather than a record of completed work. Actual dates should be filled from the team's sanctioned project planner.

| No. | Task | Effort | Exit condition |
|---|---|---|---|
| 1 | Problem definition and survey | 2 weeks | Scope and review references consolidated. |
| 2 | Requirement analysis and data schema | 2 weeks | Approved SRS and entity relationships. |
| 3 | UI design and backend skeleton | 2 weeks | Role flows and API contract agreed. |
| 4 | Data preparation and ML baseline | 2–3 weeks | Reproducible preprocessing and baseline metrics. |
| 5 | Route optimizer and travel matrix | 2–3 weeks | Feasible sample routes with test cases. |
| 6 | Application integration | 2 weeks | End-to-end workflow available in development. |
| 7 | Testing and evaluation | 2 weeks | Baseline comparison and defect log. |
| 8 | Documentation and demonstration | 1–2 weeks | Report, slides, and demo checklist. |

**Table 4.2:** Proposed development estimate.

## 4.2 PROJECT RESOURCES

Resources include three student team members, the internal guide, a development computer, internet access where required, Python and JavaScript environments, MySQL, Git, and the technical documentation for scikit-learn and OR-Tools. Responsibilities should be assigned explicitly, but each module should be reviewed by at least one other team member to reduce integration risk.

| Team member | Exam No. | Shared responsibilities |
|---|---|---|
| Bhakti Jayantilal Gundecha | 3071 | Requirements, data model, testing and report preparation; module ownership to be confirmed by the team. |
| Parth Lokhande | 1020 | System design, application integration, route workflow and documentation; module ownership to be confirmed by the team. |
| Udaykiran Bitla | 3056 | ML pipeline, UI/backend implementation and validation; module ownership to be confirmed by the team. |

**Table 4.3:** Project team and initial coordination plan.

### 4.2.1 Team Organization

### 4.2.2 Management Reporting and Communication

The team should maintain a single list of requirements, issue status, open data assumptions, test cases, and design decisions. Each review should report what was completed, what remains in progress, risks that need guidance, and evidence from tests. API contracts and database changes should be shared before integration rather than discovered only at the final demo. Decisions that affect the project scope should be confirmed with the internal guide.

## 4.3 RISK MANAGEMENT WITH RESPECT TO NP-HARD ANALYSIS

Vehicle routing with time windows is a combinatorial optimization problem. The number of possible sequences increases rapidly as the number of visit locations grows. A practical solver may use heuristics and search limits to find good feasible solutions. Consequently, project risk management must address computational time, feasible-solution detection, changing inputs, and the reliability of evaluation data.

### 4.3.1 Risk Identification and Analysis

**Table 4.4:** Project risk analysis.

| ID | Risk | Likelihood | Impact | Monitoring indicator |
|---|---|---|---|---|
| R1 | Historical data unavailable or too small | Medium | High | Missing fields, low row count, unavailable labels. |
| R2 | Priority labels inconsistent across records | Medium | High | Disagreement between business definitions and stored categories. |
| R3 | Invalid or missing coordinates | Medium | High | Failed coordinate checks or impossible distance values. |
| R4 | Route infeasible due to tight windows | Medium | High | No feasible assignment for a test instance. |
| R5 | Optimization runtime too high | Medium | Medium | Solver exceeds configured time budget. |
| R6 | Appointment changes after route generation | High | Medium | Stale plan or repeated manual edits. |
| R7 | Travel-time matrix is inaccurate | Medium | High | Large difference between estimated and observed times. |
| R8 | Class imbalance affects high-priority recall | Medium | High | Few high-priority examples or low recall. |
| R9 | API or database failure | Low/ Medium | High | Timeouts, failed writes, service unavailability. |

| ID | Risk | Likelihood | Impact | Monitoring indicator |
|---|---|---|---|---|
| R10 | Unauthorized access to operational data | Low/ Medium | High | Failed role checks or access-control defects. |
| R11 | Integration mismatch among modules | Medium | Medium | Incompatible data formats or contract changes. |
| R12 | Scope expansion reduces testing time | Medium | Medium | Repeated new features close to review deadlines. |

### 4.3.2 Risk Mitigation, Monitoring and Management

**Table 4.5:** Risk mitigation plan.

| Risk ID | Mitigation | Response if observed |
|---|---|---|
| R1–R2 | Define required fields and label meanings early; prefer approved anonymized or synthetic data for the prototype. | Record limitations; switch to a transparent ranking baseline if supervised labels are unreliable. |
| R3 | Validate latitude/longitude ranges and check for duplicates or outliers. | Reject or flag invalid records and prevent route generation until required inputs are fixed. |
| R4 | Validate appointment windows and working hours before calling the optimizer. | Return a clear infeasibility message, identify conflicting visits where possible, and allow an authorized user to review constraints. |

| Risk ID | Mitigation | Response if observed |
|---|---|---|
| R5 | Use bounded test instances and solver time limits; track computation time. | Return the best feasible solution found within the limit, or report that no feasible route was found. |
| R6 | Timestamp each generated plan and preserve appointment changes. | Mark the plan stale and request route regeneration. |
| R7 | Document the source and assumptions of all time matrices. | Re-run evaluation with an approved matrix and avoid claiming live-traffic accuracy. |
| R8 | Use per-class metrics, class weights where suitable, and cross-validation. | Report the weakness explicitly and review label quality before changing model complexity. |
| R9–R10 | Apply input validation, password hashing, role checks, restricted database access, and safe secret handling. | Log technical failures without exposing credentials; revoke invalid sessions and restore from backups where needed. |
| R11–R12 | Maintain API contracts, pull-request review, test checklists, and a frozen minimum scope for each review. | Prioritize core end-to-end workflow; defer non-essential features. |

## 4.4 PROJECT SCHEDULE

The project schedule follows the dependency chain: requirements and data design must precede model and optimizer integration; model and optimizer outputs must be stable before the dashboards can display end-to-end results; and evaluation requires the baseline and proposed approaches to use the same input records.

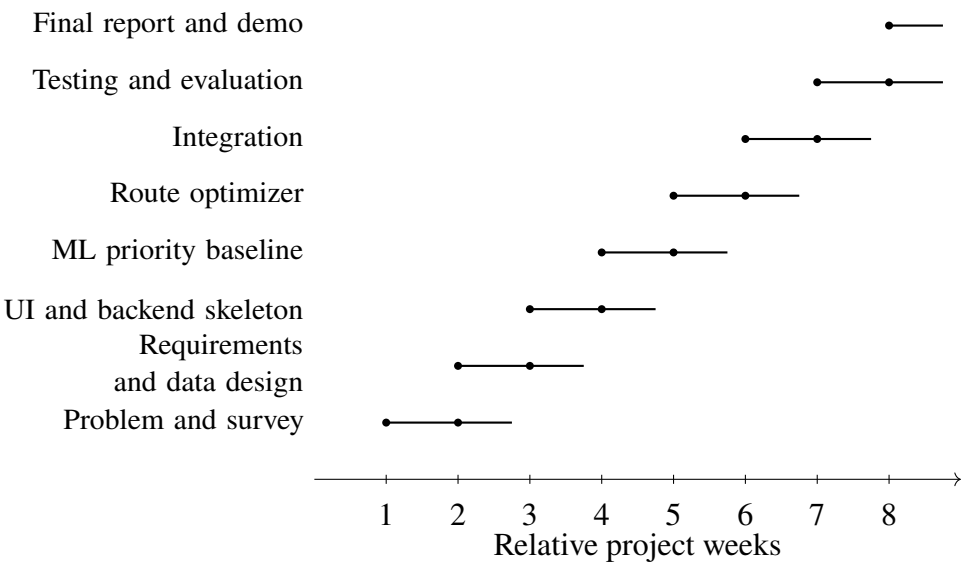



**Figure 4.1:** Indicative dependency-aware project timeline (relative weeks, not calendar dates).

## 4.5 TASK NETWORK

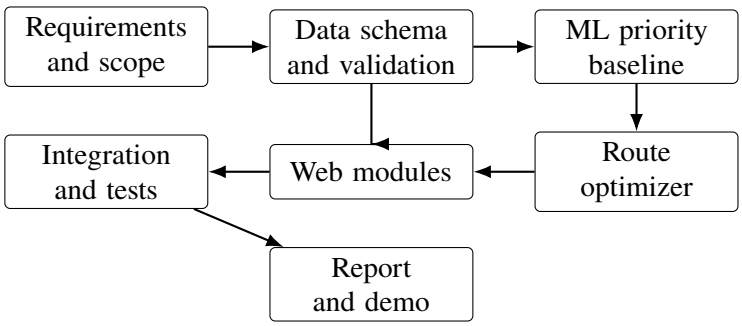



**Figure 4.2:** Project task dependency network.

# CHAPTER 5

# SOFTWARE REQUIREMENTS SPECIFICATION

## 5.1 INTRODUCTION

This Software Requirements Specification (SRS) defines the expected behaviour, interfaces, constraints, and quality attributes of the proposed application. It is intended to guide implementation and testing. Requirements are written in testable terms where possible; a requirement is not considered complete merely because a screen exists, it must also be checked with valid and invalid inputs.

## 5.2 PROJECT SCOPE

The application is a browser-accessible planning tool for authorized MRs and managers. It stores master and event data, estimates visit priority, constructs route plans, and displays progress. It supports daily planning for a selected date and territory. It is not a prescribing system and does not require patient-level clinical data.

## 5.3 USER CLASSES AND CHARACTERISTICS

### 5.3.1 Medical Representative

The MR works with an assigned territory and an assigned set of doctors. The MR needs to view the day's visits, understand priority and appointment constraints, generate or refresh a route, inspect travel and timing estimates, and update visit status and outcomes. The interface should be usable on a standard laptop browser and should avoid requiring the MR to understand optimization internals.

### 5.3.2 Manager

The manager monitors activity across representatives or territories permitted by the assigned role. Required views include planned, completed, pending, and high-priority visits; MR-wise activity; visit status; and route or summary reports. The manager may review or adjust planning inputs within authorized permissions but should not see data outside the permitted scope.

### 5.3.3 Administrator or Authorized Maintainer

An administrator manages account roles and configuration. Depending on the prototype implementation, this role may be restricted to seeded accounts and configuration tasks rather than exposed as a third full dashboard. Administrative actions must be logged and separated from normal MR actions.

## 5.4 ASSUMPTIONS AND DEPENDENCIES

- Users have valid accounts and are assigned roles and territories before using operational views.
- The master data uses stable internal identifiers for doctors and MRs.
- Time zones, date formats, and time units are standardized across the frontend, backend, and database.
- Travel-time and distance values can be supplied for all candidate arcs or an explicit fallback is available.
- A supervised ML model is only trained if labels are sufficiently reliable; otherwise the application uses a declared baseline ranking.
- The first prototype runs in a controlled environment and may use anonymized synthetic data.

## 5.5 FUNCTIONAL REQUIREMENTS

**Table 5.1:** Functional requirements and acceptance checks.

| ID | Requirement | Acceptance check |
|---|---|---|
| FR-01 | Authenticate users. | Valid credentials open the permitted view; invalid credentials do not grant access. |
| FR-02 | Enforce role-based permissions. | MR and manager accounts cannot access unauthorized screens or records through direct API calls. |
| FR-03 | Manage doctor master data. | Authorized users can create/read/update doctor fields; required fields and coordinates are validated. |
| FR-04 | Manage MR and territory data. | Working hours, territory, and start/end locations are stored and checked. |
| FR-05 | Manage visits and appointments. | Visits can be created, viewed, rescheduled, and assigned statuses with date and time constraints. |
| FR-06 | Validate data before model inference. | Missing and malformed values are reported or handled by documented rules. |
| FR-07 | Predict visit priority. | The service returns a class or score and model version for supported feature inputs. |
| FR-08 | Generate an optimized route. | The optimizer uses a time matrix, service duration, time windows, working hours, and priority-aware objective settings. |

| ID | Requirement | Acceptance check |
|---|---|---|
| FR-09 | Report infeasible route requests. | The API returns a clear status without presenting a failed route as a valid schedule. |
| FR-10 | Display the MR dashboard. | The MR sees assigned daily visits, priority, route order, estimated times, and statuses. |
| FR-11 | Update visit outcomes. | Authorized visit status and follow-up fields are saved and reflected in the dashboard. |
| FR-12 | Display manager summaries. | Aggregates include planned/completed/pending visits and priority/activity summaries for authorized scope. |
| FR-13 | Refresh plans after changes. | A changed appointment marks a prior route stale or enables route regeneration. |
| FR-14 | Compare baseline and proposed plans. | The evaluation output records metrics for identical input instances. |
| FR-15 | Preserve auditable identifiers and timestamps. | Records use stable IDs and timestamps for relevant updates and generated plans. |

## 5.6 EXTERNAL INTERFACE REQUIREMENTS

### 5.6.1 User Interfaces

The system provides a login view, an MR dashboard, doctor/appointment views, route-generation controls, visit-status controls, and a manager dashboard. Status should be communicated with text as well as colour. A generated route should show priority, stop

order, time-window information, and estimated travel, and should distinguish a confirmed plan from a stale or failed plan.

### 5.6.2 Software Interfaces

The React frontend calls FastAPI REST endpoints over HTTP(S). The backend accesses MySQL through a database client or ORM. The ML model is loaded by the backend as a versioned preprocessing/model pipeline. The routing component receives validated routing nodes, durations, a travel-time matrix, time windows, and solver configuration. External map APIs, if used, must be isolated behind an adapter so a controlled matrix can be substituted for tests.

### 5.6.3 Hardware Interfaces

No special embedded hardware is required. Standard developer or application-server hardware is sufficient for the prototype. A mobile GPS receiver is outside the initial implementation scope.

### 5.6.4 Communication Interfaces

Requests and responses use JSON over REST endpoints. Authentication tokens or secure sessions should be applied according to the deployment model. Request timeouts, input size limits, and error codes should be consistent, and sensitive content should not be written to plain-text logs.

## 5.7 NON-FUNCTIONAL REQUIREMENTS

### 5.7.1 Performance Requirements

The system should return normal CRUD and dashboard requests within a practical interactive time under the supported prototype dataset. Route generation should have a configurable upper time limit. The system must record solver duration and distinguish a feasible solution from timeout or infeasibility. Exact thresholds should be finalized after the expected dataset size is known and measured on the target environment.

### 5.7.2 Safety and Reliability Requirements

The application must not imply that an estimated route guarantees a visit or that a predicted class is a definitive business decision. It should show uncertainty or model limitations when appropriate. Invalid or conflicting appointment windows should be flagged before route generation. A failed solver response must not overwrite a previous valid route without confirmation.

### 5.7.3 Security Requirements

Passwords must be hashed using a suitable password-hashing mechanism. Endpoints must validate authenticated identity and role. Database credentials and API keys must be stored outside source control. Data access must be restricted by user role and assigned territory. Inputs are validated server-side, errors avoid leaking secrets, and logs exclude credentials and unnecessary personally identifying information.

### 5.7.4 Software Quality Attributes

The software should be maintainable through separation of frontend, API, data access, ML, and optimization modules. Testability requires deterministic sample data and fixed test matrices. Usability requires clear status feedback. Portability should be supported by environment-based configuration. Reproducibility requires versioning of model artefacts, data preparation rules, and evaluation input sets.

## 5.8 SYSTEM REQUIREMENTS

### 5.8.1 Database Requirements

The relational database stores users/roles, doctor profiles, MR profiles, visits, appointments, priority predictions, routes, and route stops. Foreign-key integrity should be used where supported. Timestamps should be stored consistently. Historical visit data should be treated as append-only where possible, with controlled updates when correcting errors.

### 5.8.2 Software Requirements

The proposed stack is React.js, Python, FastAPI, scikit-learn, Google OR-Tools, and MySQL. The precise supported versions should be pinned in dependency files before integration and deployment. Browser support should be tested against the current browser used by the target users.

### 5.8.3 Hardware Requirements

A machine with a dual-core processor, at least 4 GB RAM, and adequate storage is proposed for local development. The optimizer may need additional CPU or memory for larger routing instances. Production capacity must be chosen only after workload tests.

## 5.9 SDLC MODEL TO BE APPLIED

An iterative and incremental development model is suitable because the project contains separable modules with uncertain data and route behaviour. Each increment should produce a demonstrable component: first requirements and schema, then basic CRUD and authentication, then the ML baseline, then route generation, then dashboards and evaluation. Each increment includes tests and review. This is preferable to postponing integration until every module has been completed independently.

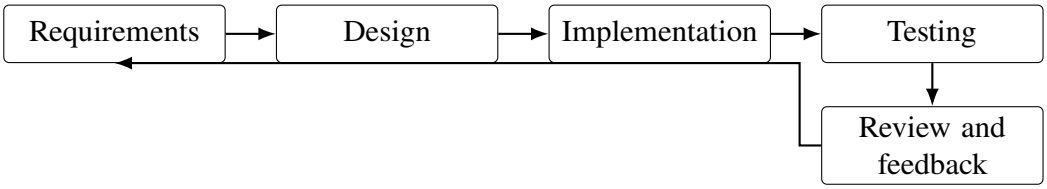



**Figure 5.1:** Iterative development model proposed for the prototype.

## 5.10 SYSTEM IMPLEMENTATION PLAN

Implementation should proceed by vertical slices so that each module is testable through the UI and API. The recommended sequence is: database schema and test fixtures; authentication and role checks; master data and appointment CRUD; priority service with a baseline model; route service and infeasibility handling; schedule and manager dashboards;

then reproducible tests and evaluation. The route service should be built against a fixed synthetic travel matrix before it depends on a live routing provider.

# CHAPTER 6

# DETAILED DESIGN

## 6.1 INTRODUCTION

This chapter describes the architecture, user behaviour, module boundaries, database entities, class relationships, and main runtime interactions. Diagrams are generated in TikZ to remain editable within this LaTeX source. They describe the proposed design and should be revised if the implementation changes.

## 6.2 ARCHITECTURAL DESIGN

The design has five logical layers: presentation, API/application services, data preparation and ML inference, constraint and route optimization, and persistence. The frontend does not train models or solve routing directly. It sends requests to the backend, which validates the request, retrieves related records, invokes the appropriate service, stores relevant results, and returns a structured response.

### 6.2.1 Presentation Layer

The presentation layer contains the login interface and role-specific dashboards. It is responsible for collecting user inputs, showing validation messages, displaying the daily route, and allowing authorized status updates. The client should not be trusted to enforce permissions; all important permission checks occur on the backend.

### 6.2.2 Application and API Layer

FastAPI endpoints coordinate user authentication, doctor and MR records, appointments, visit outcomes, priority inference, route generation, and dashboard summaries. Request and response schemas define the contract. Domain rules should be implemented in service

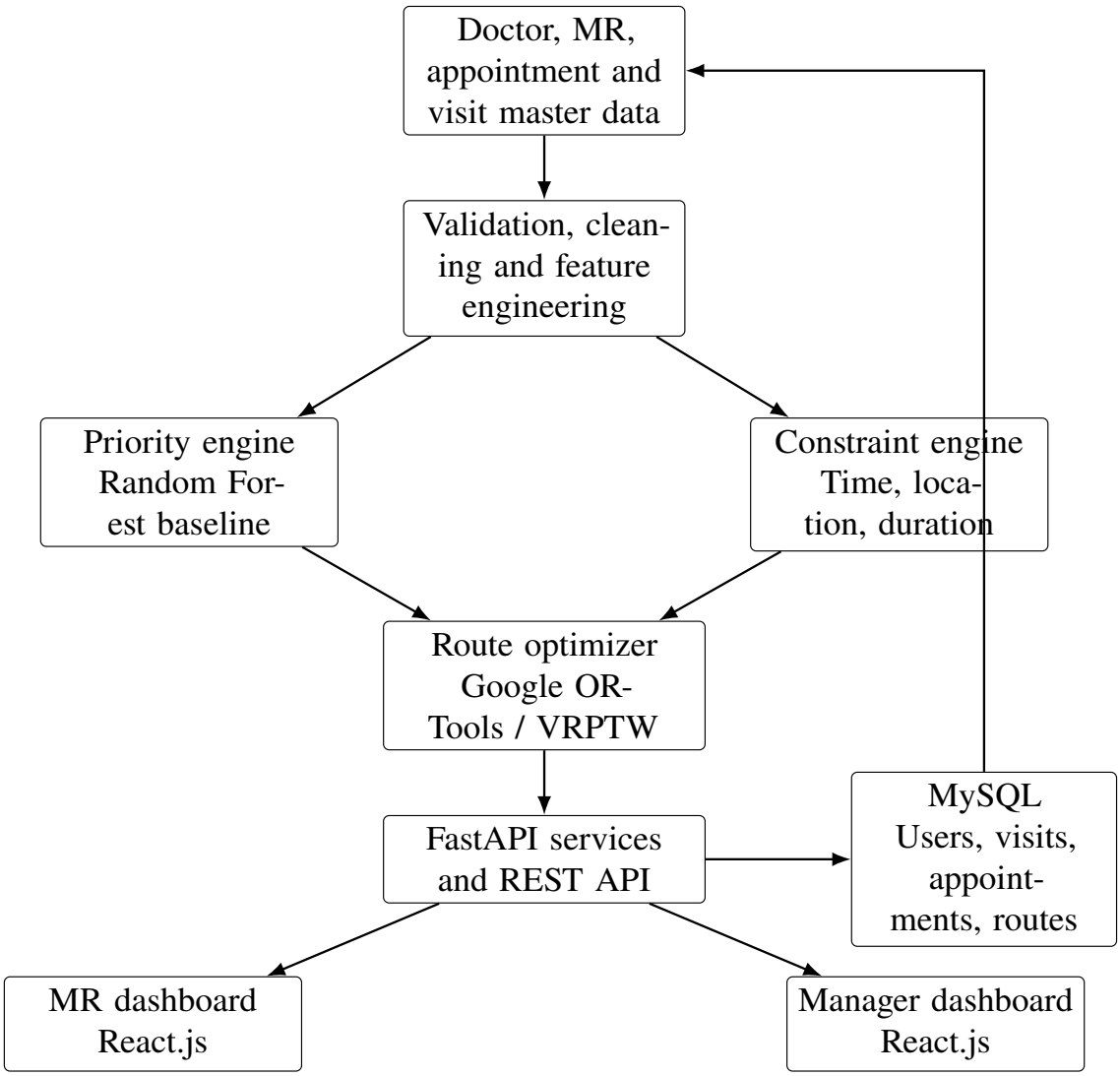



**Figure 6.1:** Overall system architecture.

functions rather than scattered throughout individual route handlers.

### 6.2.3 Machine-Learning Layer

The ML layer takes prepared features and returns a priority class or score. Preprocessing and the classifier should be saved together as a pipeline so inference receives the same transformations as training. Model version, training-data version, and timestamp should be associated with predictions. The model must not generate the route itself.

### 6.2.4 Optimization Layer

The optimization layer receives candidate locations, appointment windows, service durations, MR working time, the travel-time matrix, and optional priority penalties or rewards. It constructs a VRPTW model and searches for a feasible solution that minimizes the configured cost. It returns route stops, time estimates, objective components when available, solver status, and diagnostics. The system should distinguish infeasible input, timeout, and successful route generation.

### 6.2.5 Persistence Layer

MySQL stores operational entities and generated plans. The application should avoid storing full copies of derived information when a reference and version are sufficient. Records that support auditability, such as visit outcomes and route generation metadata, should retain timestamps and relevant status values.

## 6.3 SYSTEM WORKFLOW AND DATA FLOW

The standard MR workflow begins at login. The MR chooses a date and territory, and the application retrieves assigned doctors and appointment constraints. The backend cleans and validates the inputs, obtains or constructs a travel-time matrix, estimates visit priority, runs optimization, and returns the schedule. The MR can inspect the result and update visit status. The manager view aggregates activity for authorized records.

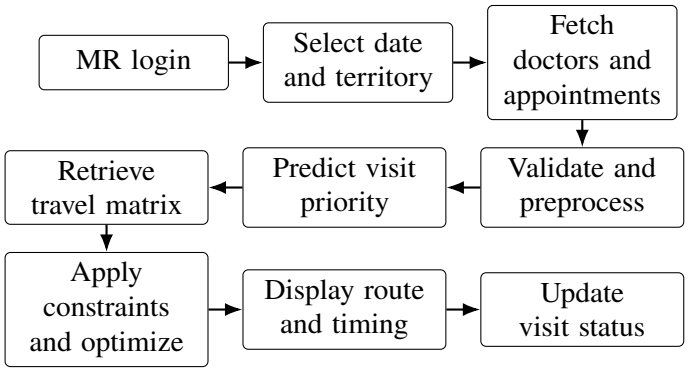



**Figure 6.2:** End-to-end MR data flow.

## 6.4 UML DIAGRAMS

### 6.4.1 Use Case Diagram

The primary actors are the Medical Representative, Manager, and Administrator. The MR manages daily execution. The manager views permitted aggregate performance and route information. The administrator manages roles and configuration. Route generation includes retrieval, validation, priority prediction, constraint assembly, optimization, and persistence.

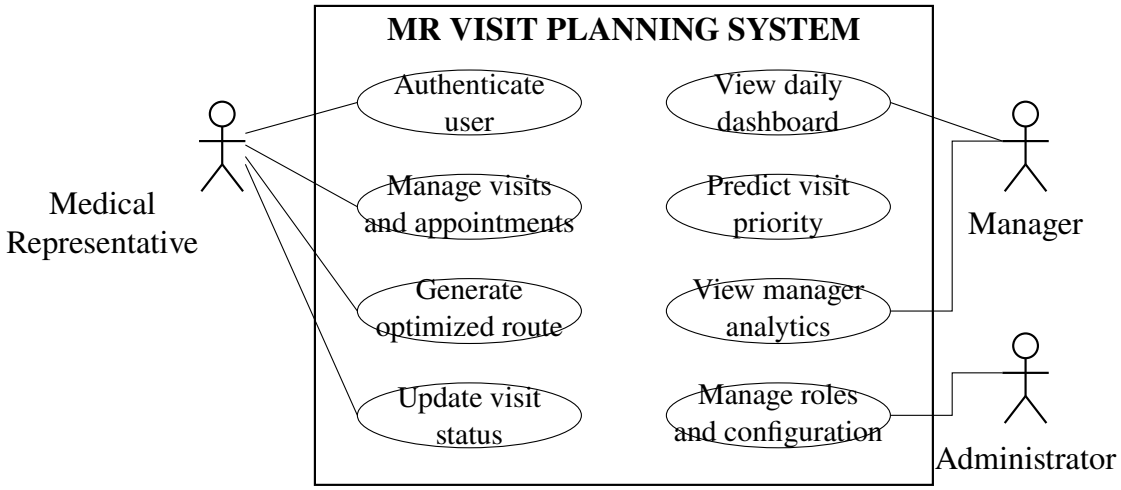



**Figure 6.3:** Use case UML diagram for the proposed application.

### 6.4.2 Activity Diagram: MR User Behaviour

The activity diagram represents the behaviour visible to an MR. The user can review the suggested route before execution. If route generation fails, the system displays a reason and leaves the previous valid route intact. A visit is marked complete only after the MR

records its status and any required outcome.

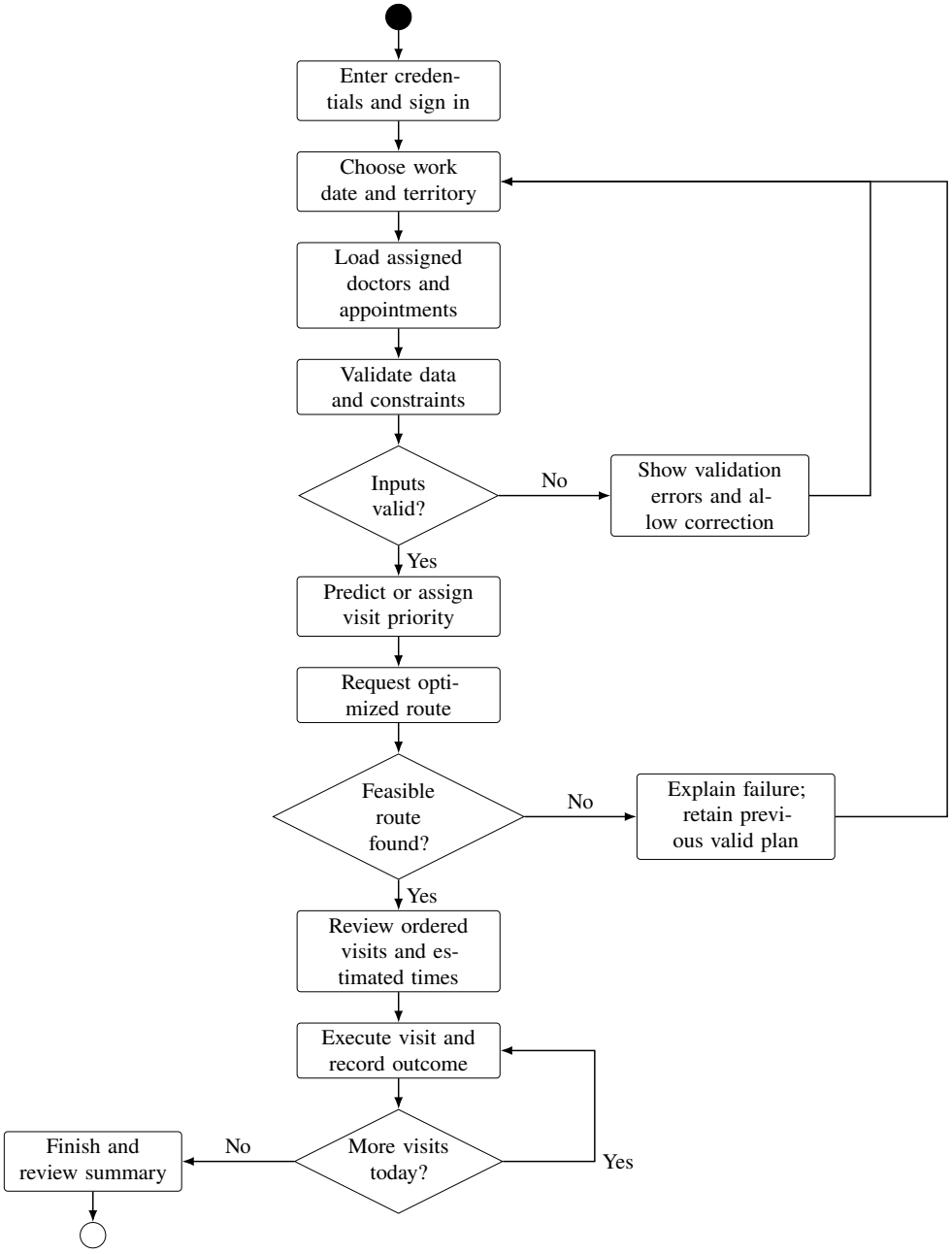



**Figure 6.4:** Activity UML diagram showing MR behaviour and exception paths.

### 6.4.3 Sequence Diagram: Route Generation

The sequence diagram clarifies the runtime messages between the user interface, API, database, priority model, and optimizer. The backend validates the request and obtains the inputs before calling either computational module. A successful route is stored and returned; failures are returned as explicit statuses.

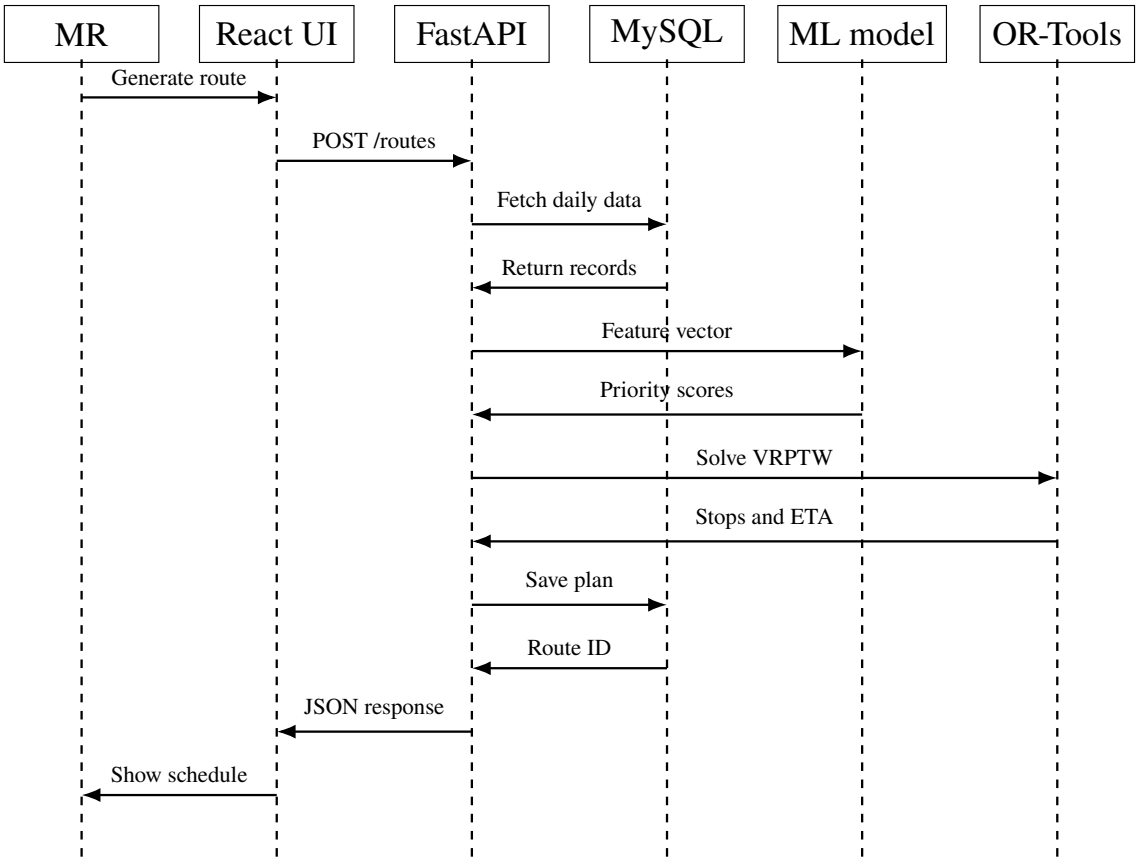



**Figure 6.5:** Sequence UML diagram for optimized route generation.

### 6.4.4 Class Diagram

The class diagram gives the principal conceptual entities. It is not a literal ORM schema; implementation may split user roles into linked profile tables or use database-specific naming. The purpose is to establish identifiers, attributes, and associations before implementing the database.

### 6.4.5 Entity–Relationship Diagram

The ER view emphasizes keys and relationships for persistent storage. A doctor can be associated with multiple appointments and visits. An MR can perform many visits and generate multiple daily route plans. A route contains an ordered set of route stops. A priority prediction is associated with a visit and should retain the model version used to generate it.

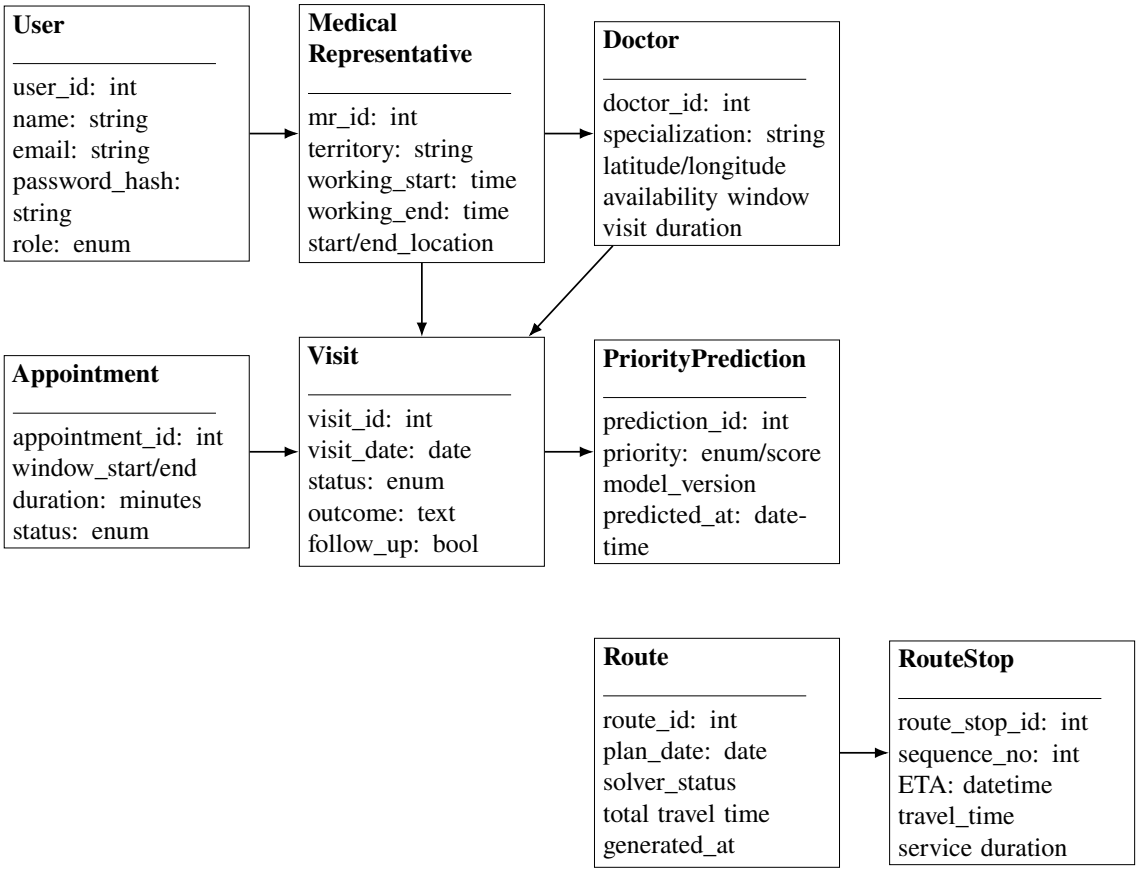



**Figure 6.6:** Conceptual UML class diagram.

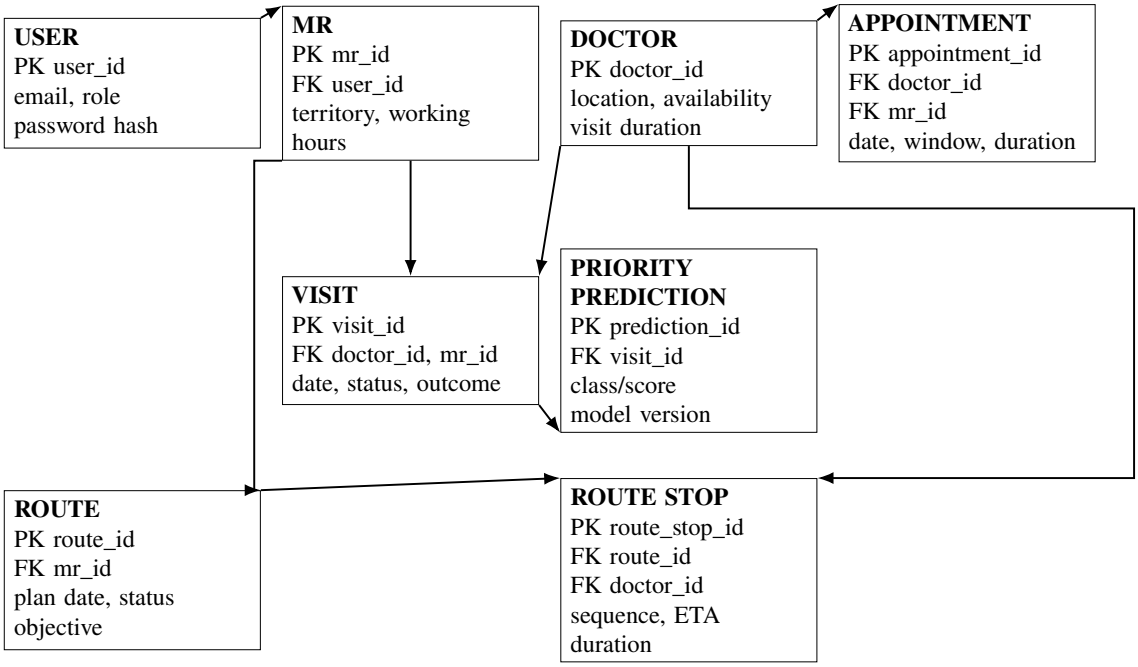



**Figure 6.7:** Conceptual entity–relationship diagram.

### 6.4.6 State Diagram: Route Lifecycle

A generated route has a lifecycle independent of visit statuses. It is initially a draft request, then either fails validation, is optimized and stored, becomes stale after an input changes, or is superseded by a regenerated route. Separating these states prevents the interface from treating a route produced with outdated appointment information as current.

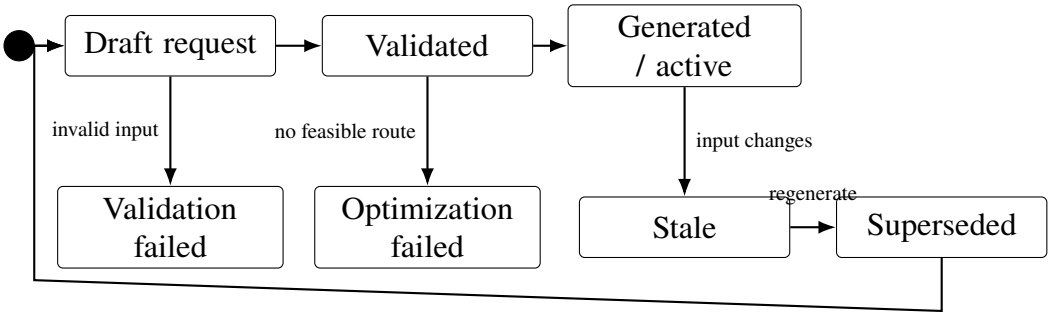



**Figure 6.8:** State UML diagram for the route lifecycle.

### 6.4.7 Component Diagram

The component view defines the principal software modules and their interfaces. The ML and optimizer are separate services at the logical level even if both are deployed inside the same backend process for the prototype.

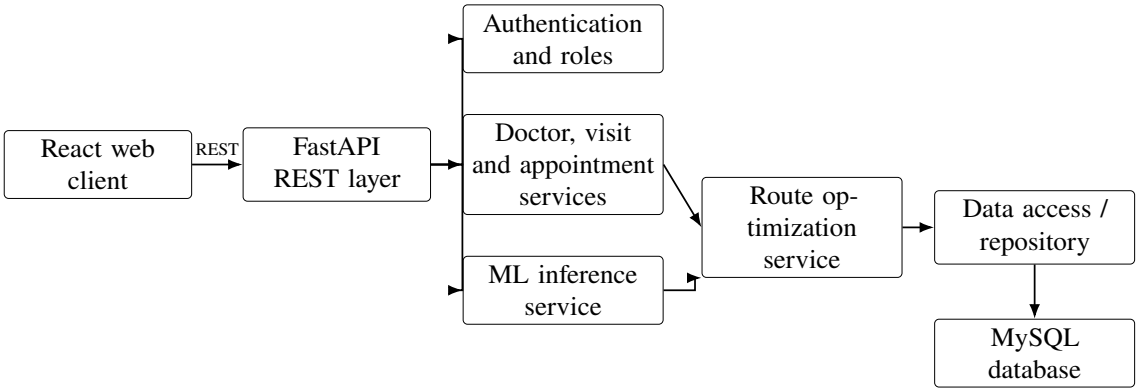



**Figure 6.9:** Component diagram of the proposed software modules.

### 6.4.8 Deployment Diagram

For the prototype, the browser runs on a user device and communicates with the application backend over a network. The backend communicates with MySQL and invokes the ML and optimization libraries in the server environment. A future production deployment may

separate these components into containers or independent services after measured load justifies it.

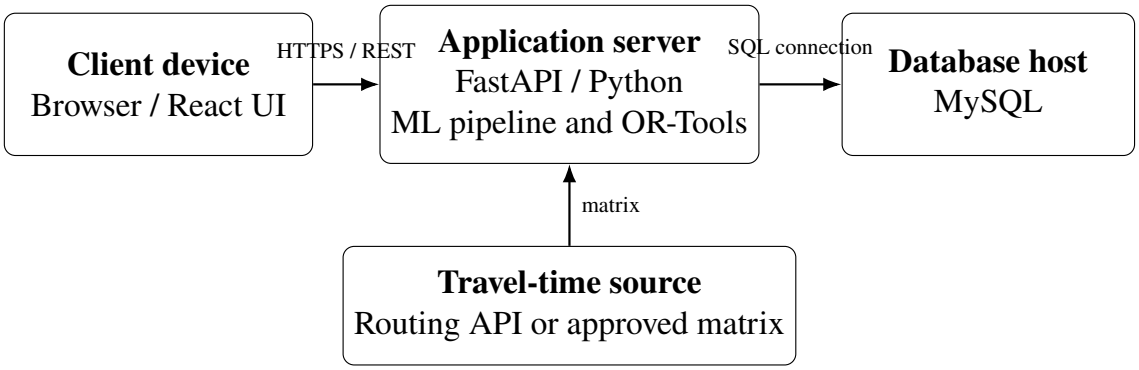



**Figure 6.10:** Deployment diagram for the academic prototype.

### 6.4.9 Data Design

The principal entities and proposed attributes are listed below. Exact column types and normalization choices should be verified during implementation.

**Table 6.1:** Proposed core data entities.

| Entity | Important attributes |
|---|---|
| Doctor | doctor_id, specialization, territory, latitude, longitude, availability_start, availability_end, typical_visit_duration, priority_category. |
| Medical Representative | mr_id, user_id, territory, working_start, working_end, start_location, end_location. |
| Visit | visit_id, doctor_id, mr_id, visit_date, planned_time, actual_arrival, actual_departure, visit_status, visit_outcome, follow_up_required. |
| Appointment | appointment_id, doctor_id, mr_id, appointment_date, window_start, window_end, duration, status. |
| Priority Prediction | prediction_id, visit_id, priority_class or score, model_version, predicted_at. |

| Entity | Important attributes |
|---|---|
| Route | route_id, mr_id, plan_date, solver_status, total_travel_time, total_distance, generated_at, model/-config version. |
| Route Stop | route_stop_id, route_id, doctor_id, sequence_number, estimated_arrival, estimated_departure, travel_time, service_duration. |
| User | user_id, display_name, email, password_hash, role, active_flag, created_at. |

### 6.4.10 Internal, Global, and Temporary Data Structures

Internal data structures include validated feature frames for ML inference, route nodes, a travel-time matrix, time-window arrays, and solver results. Global reference data includes doctor and MR identifiers, territory definitions, and allowed visit statuses. Temporary data may include a prediction batch, route candidate graph, and diagnostic messages while generating a plan. Temporary data should not be persisted unnecessarily, while outputs needed for audit and comparison should be saved with version metadata.

### 6.4.11 Database Description and Integrity Rules

Doctor and MR identifiers should be primary keys. Appointments reference both the relevant doctor and the assigned MR. Visits reference the doctor and MR and may reference an appointment. A priority prediction references the visit and model version. A route references the MR and plan date, and its ordered route-stop records reference the route and doctor. The schema should prohibit invalid foreign keys and define how deletion works; historical visits should normally be retained rather than cascade-deleted when a doctor record is deactivated.

## 6.5 MACHINE-LEARNING PRIORITY MODEL DESIGN

### 6.5.1 Feature Set

Potential features include historical visit frequency, days since last visit, previous visit outcome, follow-up requirement, existing priority category, appointment status, doctor engagement indicator, territory, and aggregate visit history. Features must be available at planning time. Any feature that reveals a future outcome or includes information recorded only after the planned visit would cause target leakage and should be excluded.

### 6.5.2 Preprocessing and Training

Preprocessing begins by checking identifiers, resolving duplicates, handling missing values with documented rules, validating categorical values, and checking time and location ranges. A train/test split should be created at the appropriate unit to avoid information leakage. If multiple rows belong to the same doctor or repeated planning period, the team should consider a grouped or time-aware split rather than randomly distributing nearly identical records across training and testing.

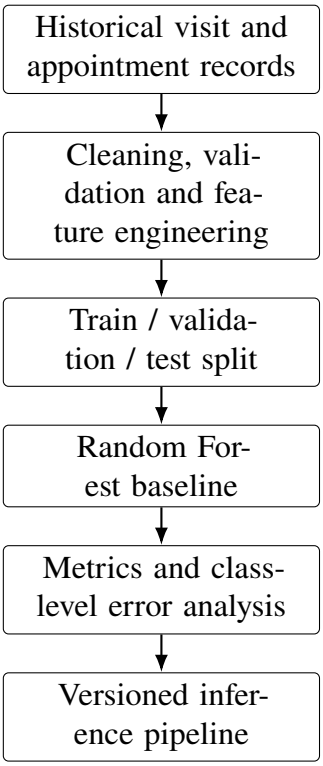



**Figure 6.11:** Proposed priority-prediction workflow.

### 6.5.3 Random Forest Formulation

A Random Forest combines multiple decision trees. If the trained model contains $T$ trees and each tree returns a class prediction $h_t(\mathbf{x})$, the classification output can be expressed as a majority vote:

$$\hat{y}(\mathbf{x}) = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \ldots, h_T(\mathbf{x})\}. \tag{6.1}$$

When calibrated class probabilities are available, a priority score could be defined as the probability of a designated class or as a business-approved weighted combination of class probabilities. The weighting must be specified with domain guidance, and it should not be selected only to make evaluation results look better.

### 6.5.4 Prediction Output and Fallback

The initial output classes are High, Medium, and Low, reflecting the illustrative categories in the review presentation. The final label schema must be aligned with the data source. If the agency has no reliable labels, a documented rule-based ranking using follow-up status, days since last visit, and appointment constraints can serve as a baseline. Such a ranking must be identified as rules-based and must not be described as a trained machine-learning result.

## 6.6 ROUTE OPTIMIZATION DESIGN

### 6.6.1 VRPTW Definition

Let $V$ be the set of nodes containing the MR start location, candidate doctor visits, and the end location. For each feasible arc $(i, j)$, let $t_{ij}$ be estimated travel time and $d_{ij}$ be distance. Each doctor visit $i$ has a service duration $s_i$ and a hard or soft time window $[a_i, b_i]$. A planned route must respect the MR's working horizon and travel/service durations. If there are multiple MRs, the model becomes a multi-vehicle routing problem.

### 6.6.2 Illustrative Objective

A simplified objective is to minimize a weighted sum of travel time, waiting time, and penalties for unserved important visits:

$$\min Z = w_t \sum_{i \in V} \sum_{j \in V} t_{ij} x_{ij} + w_w \sum_{i \in V} W_i + \sum_{i \in V} w_u(p_i) u_i, \tag{6.2}$$

where $x_{ij}$ indicates whether the route travels from node $i$ to node $j$, $W_i$ denotes waiting time, and $u_i$ indicates that visit $i$ is unserved. The penalty $w_u(p_i)$ can be larger for higher-priority visits, provided the project defines and validates that policy. This is a conceptual objective; exact variables and constraints depend on whether all visits are mandatory and whether time windows are hard or soft.

Important constraints include:

Flow conservation: $$\sum_j x_{ij} = \sum_j x_{ji} \quad \text{for each intermediate node } i, \tag{6.3}$$

Visit coverage: $$\sum_j x_{ij} + u_i = 1 \quad \text{for each candidate visit } i, \tag{6.4}$$

Time propagation: $$T_j \geq T_i + s_i + t_{ij} - M(1 - x_{ij}), \tag{6.5}$$

Time windows: $$a_i \leq T_i \leq b_i \quad \text{for mandatory hard-window visits,} \tag{6.6}$$

Work horizon: $$H_{\text{start}} \leq T_i \text{ and route end} \leq H_{\text{end}}. \tag{6.7}$$

The large constant $M$ is a modelling device for conditional time propagation. A practical implementation should use solver dimensions and domain bounds where possible rather than an unnecessarily large arbitrary constant.

### 6.6.3 Illustrative Appointment Inputs

A high-priority visit can be scheduled later if its available time window requires it. A feasible schedule must account for travel and service time between all stops. The final stop order cannot be inferred from priority alone; it must be computed from the complete time and distance matrix, visit durations, windows, working hours, and objective weights.

| Doctor | Illustrative priority | Time window |
|---|---|---|
| A | High | 09:00–10:00 |
| B | Medium | 10:00–12:00 |
| C | High | 13:00–14:00 |
| D | Low | 14:00–17:00 |

**Table 6.2:** Illustrative inputs from the design presentation; not experimental results.

### 6.6.4 Travel-Time Matrix

The optimizer requires a matrix of travel-time estimates, and optionally a distance matrix. The matrix may come from an approved routing API, agency-provided information, precomputed distances, or a clearly labelled controlled simulation. In experiments, the same matrix must be used when comparing baseline and proposed routes. If live traffic is unavailable, travel-time results must be described as estimated rather than real-time.

### 6.6.5 Infeasibility Handling

Before optimization, the backend should validate non-empty candidate sets, all required time windows, non-negative service durations, MR working hours, and matrix dimensions. When a route cannot be generated, the system should record solver status and show an understandable message. It may identify restrictive inputs, but it should not silently discard a visit or violate a hard window. Any optional-visit or soft-window policy must be explicit in the request configuration.

## 6.7 MODULE DESIGN

**Table 6.3:** Functional module breakdown.

| No. | Module | Responsibilities |
|---|---|---|
| 1 | Authentication and role management | Login, session/token validation, role and territory permissions. |

| No. | Module | Responsibilities |
|---|---|---|
| 2 | Doctor management | Doctor master, specialization, territory, location, availability and typical duration. |
| 3 | MR management | Profile, territory, working time and route start/end locations. |
| 4 | Visit and appointment management | Create/reschedule visits, appointment windows, status, outcomes and follow-up. |
| 5 | ML priority engine | Feature extraction, model loading, inference, output validation and versioning. |
| 6 | Route optimization | Matrix validation, time constraints, objective setup, OR-Tools invocation and infeasibility handling. |
| 7 | MR dashboard | Daily visit list, priority, route, schedule and visit status. |
| 8 | Manager dashboard | MR-wise activity, planned/completed/pending visits, high-priority counts and route summaries. |
| 9 | Evaluation and reports | Baseline comparison, metric computation, experiment metadata and exportable summaries. |

## 6.8 ILLUSTRATIVE DASHBOARD BEHAVIOUR

The MR dashboard should show total assigned visits, completed visits, pending visits, high-priority visits, and a route view. A route stop should include doctor identifier, visit order, expected arrival or appointment window, priority, travel estimate, and status. The manager dashboard should show aggregate counts for authorized MRs and identify any value that has not yet been calculated. The review presentation includes illustrative numbers such as 12 MRs, 82 planned visits, 61 completed, 21 pending, and 19 high-priority visits.

These are demo placeholders and must not be reported as measured project results unless reproduced from a verified dataset.

## 6.9 API DESIGN OVERVIEW

The following endpoints are illustrative contracts for the intended architecture. They should be refined in implementation and documented using generated OpenAPI specifications.

**Table 6.4:** Indicative REST API endpoints.

| Method | Endpoint | Purpose | Role |
|---|---|---|---|
| POST | /auth/login | Authenticate user | Public |
| GET | /doctors | List permitted doctors | MR/ Manager |
| POST | /visits | Create a visit | MR/ Manager |
| GET | /visits/daily | Retrieve daily plan candidates | MR |
| POST | /priorities/predict | Return visit priority | MR/ Manager service |
| POST | /routes/generate | Generate a route plan | MR/ Manager |
| GET | /routes/{id} | Retrieve route and stops | Authorized user |
| PATCH | /visits/{id}/status | Update status/outcome | MR |
| GET | /manager/summary | Retrieve authorized aggregates | Manager |
| GET | /reports/evaluation | Retrieve experiment metrics | Authorized reviewer |

# CHAPTER 7

# OTHER SPECIFICATIONS

## 7.1 ADVANTAGES

1. **Integrated planning:** priority, appointments, and travel constraints are considered within one workflow.
2. **More consistent prioritization:** a trained model or explicitly defined fallback rules use the same criteria across records.
3. **Feasibility-aware schedules:** time windows, service duration, and MR working hours are modelled in route generation.
4. **Operational visibility:** the MR sees a daily schedule while the manager sees permitted summary and activity information.
5. **Measurable outcomes:** prediction and route quality can be compared with baselines using recorded metrics.
6. **Modular implementation:** web interface, database, ML, and optimizer can be tested independently.

## 7.2 LIMITATIONS

1. ML performance depends on the amount, reliability, and representativeness of historical data and priority labels.
2. An initial estimated travel-time matrix may not reflect current traffic, closures, or parking time.
3. Route generation can become difficult as the number of visits and constraints grows; a solver may return a feasible but not globally optimal solution.

4. Appointment changes after route generation can make the current plan stale and require re-optimization.

5. The initial prototype may cover only a limited territory and may not include multiple-city operations or commercial CRM integration.

6. Manager dashboards are only as accurate as the visit status and operational information entered by users.

7. Model and route performance remain to be validated with experimental data; no improvement percentage is claimed in this report.

### 7.3 SECURITY AND PRIVACY

The core system requires doctor-visit scheduling information, not patient medical records. Internal identifiers such as D001 and MR007 should be used in experimental datasets instead of unnecessary personal details. Access should be restricted to information required by the user's role and territory. Passwords should be stored as salted hashes; secrets should be excluded from source control; input validation should be performed on the backend; and database access should use least-privilege credentials.

Audit fields should record route generation time, model version, solver status, and relevant input/version identifiers so that a schedule can be reproduced. Logs must not expose passwords, tokens, or sensitive personal data. Data retention, consent, and legal obligations should be reviewed by the institution and any participating organization before using real operational data.

### 7.4 TESTING STRATEGY

Testing should be applied at unit, integration, system, usability, and evaluation levels. Unit tests cover time-window validation, feature preprocessing, prediction output schema, travel-matrix dimensions, and route-result parsing. Integration tests cover frontend-to-backend calls, persistence, model inference, and the optimizer. System tests cover representative end-to-end user journeys. Evaluation tests run baseline and proposed methods against the

same instances.

| Test category | Example | Expected behaviour |
|---|---|---|
| Input validation | Appointment end time earlier than start time | Request rejected with a clear error. |
| Authorization | MR requests a manager-only endpoint | Access denied and no unauthorized data returned. |
| ML pipeline | Valid feature vector supplied to model | Output matches the agreed schema and class domain. |
| Routing | Small matrix with feasible windows | Route contains ordered stops and feasible timing. |
| Routing failure | Windows make all schedules infeasible | Solver status displayed; no false route presented. |
| Status update | MR records a completed visit | Database and dashboard show the updated status. |
| Stale plan | Appointment changes after generation | Previous route is marked stale or requires confirmation. |
| Baseline comparison | Same input sent to both planners | Metrics are computed from the same source data. |

**Table 7.1:** Illustrative test plan.

## 7.5 EVALUATION PLAN

### 7.5.1 ML Evaluation

For classification, accuracy is the fraction of correct predictions. Precision measures the proportion of predicted positive cases that are correct, and recall measures the proportion of actual positive cases found by the model. The F1-score is the harmonic mean of precision and recall:

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{7.1}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{7.2}$$

$$F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{7.3}$$

Metrics should be reported by priority class. Macro-averaged scores can help reveal performance that is hidden by a dominant class. A confusion matrix is useful for checking

whether high-priority visits are incorrectly classified as low priority. ROC-AUC should be used only when the class structure and evaluation setup support it; precision-recall measures may be more informative under class imbalance [16, 17, 18].

### 7.5.2 Route Evaluation

The baseline may be the existing manual plan, a fixed priority ordering, nearest-neighbour routing, or another agreed simple heuristic. The proposed approach combines predicted priority with OR-Tools. The same set of candidates and same travel-time input should be provided to both approaches where possible. Metrics include total distance, total travel time, number of visits completed, time-window violations, waiting time, unserved visits, and route computation time. Baseline values must be measured; placeholder symbols such as X and Y are not results.

## 7.6 APPLICATIONS

The main application is planned daily activity management for Medical Representatives. The same general architecture may support other field-service settings where workers visit multiple locations with different importance levels, appointment windows, and service durations. Examples include maintenance technicians, territory sales personnel, inspection teams, and scheduled support visits. Transfer to these areas requires a different feature model, constraints, user roles, and privacy review.

## 7.7 FUTURE SCOPE

Possible extensions include real-time GPS tracking, dynamic traffic integration, route re-optimization during the day, a mobile application, multiple-city deployment, simultaneous optimization for several MRs, CRM integration, automatic appointment rescheduling, stronger ranking models, and advanced optimization methods. Predictive analytics for field performance could be introduced only after defining valid measures and ensuring that such analytics are appropriate for the users and data. Future work should prioritize reliability and transparency before increasing model complexity.

| Metric | Baseline | Proposed | Interpretation |
|---|---|---|---|
| Total distance | To measure | To measure | Lower is generally preferable if visit coverage is maintained. |
| Total travel time | To measure | To measure | Lower reduces time spent travelling. |
| Visits completed | To measure | To measure | Higher is preferable when visit quality and feasibility remain acceptable. |
| Time-window violations | To measure | To measure | Zero is required for hard-window feasibility. |
| Waiting time | To measure | To measure | Lower can improve schedule efficiency. |
| Unserved high-priority visits | To measure | To measure | Lower indicates stronger coverage of important visits. |
| Route computation time | To measure | To measure | Must remain acceptable for interactive use. |

**Table 7.2:** Baseline and proposed-system comparison framework.

# CHAPTER 8

# SUMMARY AND CONCLUSION

## 8.1 SUMMARY

This report defines an AI-based system for Medical Representative visit prioritization and route optimization. The key design decision is to separate two connected tasks: a machine-learning model estimates the importance of each planned visit, and a constraint-based optimizer determines a feasible sequence using travel time, appointment windows, visit duration, and MR working hours. The system combines these outputs in a web application with separate MR and manager dashboards.

The report covered existing manual planning limitations, a review of routing and physician-detailing research, the proposed system scope, development plan, software requirements, architecture, data entities, UML diagrams, module boundaries, testing strategy, security, privacy, risks, and evaluation metrics. The design proposes React.js, FastAPI/Python, scikit-learn, Google OR-Tools, and MySQL. It also recognizes that accurate predictions depend on reliable labels and that route quality depends on a well-defined travel-time matrix.

## 8.2 CONCLUSION

The proposed workflow offers a structured way to combine historical visit information with operational constraints. It is expected to make the planning process more transparent and easier to evaluate than a purely manual sequence, but this expectation must be verified. At the current design stage, the system architecture, modules, intended data flow, and evaluation approach are specified, while database implementation, ML pipeline, routing, integration, and testing remain work items as recorded in the review presentation. No numerical model-performance or travel-reduction claims are made until experiments are complete.

## 8.3 NEXT STEPS

The next implementation steps are to finalize the data dictionary and priority-label definition, prepare a safe test dataset, build a database-backed CRUD foundation, implement the baseline model, integrate a controlled travel-time matrix with OR-Tools, test route feasibility and error handling, and connect the results to the MR and manager dashboards. Once the end-to-end workflow is stable, both ML and route quality should be measured against baselines and recorded in the final report.

# BIBLIOGRAPHY


[1] G. B. Dantzig and J. H. Ramser, "The Truck Dispatching Problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959, doi: 10.1287/mnsc.6.1.80.

[2] G. Clarke and J. W. Wright, "Scheduling of Vehicles from a Central Depot to a Number of Delivery Points," *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964, doi: 10.1287/opre.12.4.568.

[3] M. M. Solomon, "Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints," *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987, doi: 10.1287/opre.35.2.254.

[4] M. M. Solomon and J. Desrosiers, "Survey Paper: Time Window Constrained Routing and Scheduling Problems," *Transportation Science*, vol. 22, no. 1, pp. 1–13, 1988, doi: 10.1287/trsc.22.1.1.

[5] P. Toth and D. Vigo, eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2014.

[6] G. Laporte, "The Vehicle Routing Problem: An Overview of Exact and Approximate Algorithms," *European Journal of Operational Research*, vol. 59, no. 3, pp. 345–358, 1992, doi: 10.1016/0377-2217(92)90192-C.

[7] O. Bräysy and M. Gendreau, "Vehicle Routing Problem with Time Windows, Part I: Route Construction and Local Search Algorithms," *Transportation Science*, vol. 39, no. 1, pp. 104–118, 2005, doi: 10.1287/trsc.1030.0056.

[8] O. Bräysy and M. Gendreau, "Vehicle Routing Problem with Time Windows, Part II: Metaheuristics," *Transportation Science*, vol. 39, no. 1, pp. 119–139, 2005.

[9] D. Pisinger and S. Ropke, "A General Heuristic for Vehicle Routing Problems," *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007, doi: 10.1016/j.cor.2005.09.012.

[10] V. Pillac, M. Gendreau, C. Guéret, and A. L. Medaglia, “A Review of Dynamic Vehicle Routing Problems,” *European Journal of Operational Research*, vol. 225, no. 1, pp. 1–11, 2013, doi: 10.1016/j.ejor.2012.08.015.

[11] T. Bektaş and G. Laporte, “The Pollution-Routing Problem,” *Transportation Research Part B: Methodological*, vol. 45, no. 8, pp. 1232–1250, 2011, doi: 10.1016/j.trb.2011.02.004.

[12] J. C. Yi, G. Anandalingam, and L. A. Sorrell, “An Expert System to Physician-Detailing Planning,” *Expert Systems with Applications*, vol. 25, no. 4, pp. 533–544, 2003, doi: 10.1016/S0957-4174(03)00093-9.

[13] J. C. Yi, “Knowledge-Based Approach to Improving Detailing Plan in Multiple Product Situations Using PDE Weights,” *Expert Systems with Applications*, 2011, doi: 10.1016/j.eswa.2010.09.044.

[14] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[15] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[16] T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[17] J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240, doi: 10.1145/1143844.1143874.

[18] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[19] X. Liu, Y.-L. Chen, L. Y. Por, and C. S. Ku, “A Systematic Literature Review of Vehicle Routing Problems with Time Windows,” *Sustainability*, vol. 15, no. 15, article 12004, 2023, doi: 10.3390/su151512004.

[20] Google for Developers, “Vehicle Routing Problem with Time Windows,” OR-Tools Documentation. Available: https://developers.google.com/optimization/routing/vrptw. Accessed: 9 October 2026.

[21] Meta Open Source, “React Documentation.” Available: https://react.dev/. Accessed: 9 October 2026.

[22] S. Ramírez, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/. Accessed: 9 October 2026.

[23] Oracle, “MySQL Reference Manual.” Available: https://dev.mysql.com/doc/. Accessed: 9 October 2026.

# CHAPTER A

# PROJECT PLANNER

The following checklist reflects the proposed work breakdown. The team should update the status and actual dates during each review.

**Table A.1:** Project planner checklist.

| No. | Deliverable | Planned phase | Status at design review |
|---|---|---|---|
| 1 | Problem definition | Semester I | Completed |
| 2 | Literature survey | Semester I | Completed |
| 3 | Requirement analysis | Semester I | Completed / refine as needed |
| 4 | System architecture | Semester I | Completed |
| 5 | Technology selection | Semester I | Completed |
| 6 | Database design | Semester I | In progress |
| 7 | UI design | Semester I | Completed |
| 8 | Backend structure | Semester II | In progress |
| 9 | ML pipeline and baseline | Semester II | In progress |
| 10 | OR-Tools optimization | Semester II | In progress |
| 11 | Dashboard integration | Semester II | In progress |
| 12 | System and security testing | Semester II | Upcoming |
| 13 | Evaluation and comparison | Semester II | Upcoming |
| 14 | Deployment and final demo | Semester II | Upcoming |

# CHAPTER B

# REVIEWER COMMENTS AND ACTION LOG

The supplied project material does not contain a completed reviewer-comments sheet. Use the table below to record feedback at the next review rather than leaving the approval status implicit.

**Table B.1:** Reviewer comment and action log.

| No. | Comment / required change | Action taken / evidence | Status |
|---|---|---|---|
| 1 | To be filled after review. | To be filled by the team. | Pending |
| 2 | To be filled after review. | To be filled by the team. | Pending |
| 3 | To be filled after review. | To be filled by the team. | Pending |

# CHAPTER C

# PLAGIARISM REPORT

Attach the institutional plagiarism-check report in the final submission. The report was not included among the provided source files, so no similarity percentage or approval status is asserted here. Replace this page with the authorized report when available.